

Как работают компоненты и пропсы в Астро

Оглавление

- Как работают компоненты и пропсы в Астро
 - Оглавление
- Проблема
- Цель
- Решение
 - Пропсы компонента
 - Компоненты
 - Структура компонента
 - Скрипт компонента
 - Шаблон компонента
 - Компонентный дизайн

- Слоты
- Именованные слоты
- Резервный контент для слотов
- Перенос слотов
- HTML-компоненты
- Дополнительно (глубже в кроличью нору)
 - Fragments
 - set:directives
 - Справочник директив шаблонов
 - Правила
 - Общие директивы
 - class:list
 - set:html
 - set:text
 - client:directives
 - client:load

- client:idle
- timeout
- client:visible
- client:visible=
{{rootMargin}}
- client:media
- client:only
- Отображение
загружаемого
содержимого
- Индивидуальные
клиентские
директивы
- Директивы
сервера
- server:defer
- Директивы по
скрипту и стилю
- is:global
- is:inline
- define:vars

- Расширенные директивы
- is:raw
- framework-components
 - Использование компонентов фреймворка
 - Увлажняющие интерактивные компоненты
 - Доступные рекомендации по поддержанию баланса загрузки
 - Смешивание фреймворков
 - Передача свойств компонентам фреймворка
- Передача дочерних элементов

компонентам
фреймворка.

- Компоненты
фреймворка
вложенности
- Можно ли
использовать
компоненты
Astro внутри
компонентов
моего
фреймворка?
- Можно ли hydrate
компоненты
Astro?
- Сериализация
- Неподдерживаемые
структуры данных,
передаваемые
компонентам, такие как
функции

- Суть проблемы:
Граница между
сервером и
клиентом
- Почему функции
"не переезжают"
из мира 1 в мир 2?
- Объяснение
фразы
"человеческим
языком"
- Это значит:
- Аналогия с
переездом:
- Astro-syntax
 - Выражения,
подобные **JSX**
 - Переменные
 - Динамические
атрибуты

- Динамический HTML
- Динамические теги
- Фрагменты
- Различия между Astro и JSX
- Атрибуты
 - Несколько элементов
 - Комментарии
 - Вспомогательные утилиты компонентов
 - Astro.slots
- Astro.slots.has()
 - Astro.slots.render()
 - Astro.self
- passing-components-to-mdx-content

- @astrojs/mdx
 - Почему именно **MDX**?
 - Установка
 - Ручная установка
 - Интеграция с редактором
 - Использование MDX в Астро
 - Использование MDX с коллекциями контента
 - Использование экспортированных переменных в MDX
 - Экспортируемые объекты недвижимости

- Использование переменных метаданных в MDX
- Использование компонентов в **MDX**
- Назначение пользовательских компонентов элементам **HTML**
- Передача **components** содержимого в формат **MDX**
- Конфигурация
- Параметры, унаследованные от конфигурации **Markdown.**

- extendMarkdownC
onfig
 - recmaPlugins
 - optimize
 - ignoreElementNa
mes
- requestIdleCallback
- requestIdleCallback-
method
 - callback
 - options
 - Необязательный
 - Возвращаемое
значение
- addclientdirective-option
 - Простыми
словами:
 - Зачем это нужно:
 - Тип: (directive:
ClientDirectiveCon
fig) => void;

- Пример использования:
- Переделанный пример с выводом в консоль
- Как это использовать в компоненте
- типы
- `server-islands`
 - Зачем это нужно (простыми словами)
 - Серверные острова
 - Компоненты серверного острова
 - Передача ресурсов на

серверные
острова.

- Резервный контент для серверного острова
- Как это работает
- Кэширование
- Доступ к URL-адресу страницы в изолированном серверном модуле.
- Повторное использование ключа шифрования
- Стили и CSS
 - Стилизация в Astro
 - Локальные стили

- global-styles
- Глобальные стили
- Комбинирование классов с class:list
- CSS-переменные
- Передача **class** дочернему компоненту
- Встроенные стили
- Внешние стили
- Импорт локальной таблицы стилей
- Импорт стилей из **npm**-пакета
- Подключение статической таблицы стилей через тег **<link>**
- Порядок наследования

- Локальные стили
- Порядок импорта
- Подключение стилей через теги `<link>`
- Tailwind
- Добавление Tailwind 4
- CSS-препроцессоры
 - Sass и SCSS
 - Stylus
 - Less
 - LightningCSS
- В компонентах фреймворков
 - PostCSS
- Фреймворки и библиотеки
 - React / Preact

- Vue
- Svelte
- Стилизация
Markdown
- Продакшен
- Контроль сборки
- Продвинутые
настройки
- Импорт **CSS** с ?
raw
- Импорт CSS с ?url
- Скрипты с : async, defer
 - defer
 - async
 - Динамически
загружаемые
скрипты
- Скрипты и обработка
событий
 - Скрипты на
стороне клиента

- client-side-scripts
- Обработка скриптов
- Необработанные скрипты
- Включите файлы **JavaScript** на свою страницу.
- Импорт локальных скриптов
- Загрузка внешних скриптов
- Общие шаблоны скриптов
- Обработка **onclick** и другие события
- Веб-компоненты с пользовательскими элементами

- Передайте переменные метаданных скриптам.
- `markdown-content`
- `hydration`
- `XSS`
- [Материалы для статьи](#)

Проблема

передача информации между файлами в Astro

Цель

- Разобраться с передачей пропсов

- Лучшие способы передачи
- Разбор компонентов в пропсах

Решение

Пропсы компонента

Компонент **Astro** может определять и принимать входные параметры — пропсы. Эти пропсы затем становятся доступными шаблону компонента для рендеринга HTML. Пропсы доступны в глобальном объекте ***Astro.props*** в вашем скрипте.

Вот пример компонента, который получает пропсы **greeting** и **name**. Обратите внимание, что получаемые входные параметры деструктурированы из глобального объекта ***Astro.props***.

```
---  
// Применение:  
<GreetingHeadline  
  greeting="Привет"  
  name="Партнёр" />  
const { greeting, name } =  
  Astro.props;  
---  
<h2>{greeting}, {name}!  
</h2>
```

Этот компонент при импорте и визуализации в других компонентах **Astro**, макетах или страницах может передавать эти параметры в качестве атрибутов:

```
---  
import GreetingHeadline  
from
```

```
'./GreetingHeadline.astro';  
const name = "Astro"  
---  
<h1>Поздравительная  
открытка</h1>  
<GreetingHeadline  
greeting="Привет" name=  
{name} />  
<p>Надеюсь, у вас чудесный  
день!</p>
```

Вы также можете определить свои пропсы с помощью **TypeScript** с интерфейсом типа **Props**. Astro автоматически подхватит интерфейс **Props** в вашем блоке метаданных и выдаст предупреждения/ошибки типа. Этим входным параметрам также могут быть присвоены значения по умолчанию при деструктуризации из ***Astro.props***.

```
---  
interface Props {  
  name: string;  
  greeting?: string;  
}  
  
const { greeting =  
  "Привет", name } =  
  Astro.props;  
---  
<h2>{greeting}, {name}!  
</h2>
```

Пропсам компонента могут быть присвоены значения по умолчанию, которые можно использовать, если они не указаны.

```
---  
const { greeting =
```

```
"Привет", name =  
"Астронавт" } =  
Astro.props;  
---  
<h2>{greeting}, {name}!  
</h2>
```

Компоненты

Структура компонента

Компонент **Astro** состоит из двух основных частей: Скрипта компонента и Шаблона компонента. Каждая часть выполняет свою работу, но вместе они обеспечивают структуру, которая одновременно проста в использовании и достаточно выразительна, чтобы справиться с тем, что вы захотите создать.

```
---  
// Скрипт компонента  
(JavaScript)  
---  
<!-- Шаблон компонента  
(HTML + выражения JS) -->
```

Скрипт компонента

Astro использует блок кода (**---**) для идентификации скрипта в вашем компоненте **Astro**. Если вы когда-либо писали Markdown раньше, возможно, вы уже знакомы с похожей концепцией, называемой метаданные. Идея **Astro** о скрипте компонента была непосредственно вдохновлена этой концепцией.

Вы можете использовать скрипт компонента для написания любого кода **JavaScript**, необходимого для визуализации шаблона. Это может включать в себя:

импорт других компонентов **Astro** импорт компонентов из других фреймворков, таких как **React** импорт данных, например файла **JSON** получение контента из **API** или базы данных создание переменных, на которые вы будете ссылаться в своем шаблоне

```
---  
import SomeAstroComponent  
from  
'../components/SomeAstroCom  
ponent.astro';  
import SomeReactComponent  
from
```



```
'../components/SomeReactComponent.jsx';
import someData from
'../data/pokemon.json';

// Доступ к пропсам
компонента, таким как `
```

Блок кода предназначен для того, чтобы гарантировать, что написанный в нём

JavaScript остаётся «изолированным». Он не попадёт в ваше фронтенд-приложение и не окажется в руках пользователя. Здесь можно безопасно писать код, который является ресурсоёмким или чувствительным (например, подключение к вашей приватной базе данных), не беспокоясь о том, что он когда-либо попадёт в браузер пользователя.

Шаблон компонента

Шаблон компонента следует за блоком кода и определяет вывод **HTML** вашего компонента.

Если вы напишете здесь простой **HTML**, ваш компонент будет отображать этот **HTML** на любой странице Astro, в которую он импортирован и используется.

Однако, синтаксис **Astro** так же поддерживает **JavaScript** выражения, теги **<style>** и **<script>** **Astro**, импортированные компоненты, и специальные директивы **Astro**. Данные и значения, определённые в скрипте компонента, можно использовать в шаблоне компонента для создания динамически создаваемого **HTML**.

```
---  
// Здесь ваш скрипт  
компонента!  
import Banner from  
'../components/Banner.astro'  
;  
import Avatar from  
'../components/Avatar.astro'  
;  
import  
ReactPokemonComponent from  
'../components/ReactPokemon
```

```
Component.jsx';  
const myFavoritePokemon =  
  [/* ... */];  
const { title } =  
  Astro.props;  
---  
<!-- Поддерживаются HTML  
комментарии! -->  
{/* Синтаксис комментариев  
JS также действителен! =>  
скорее jsx */}  
  
<Banner />  
<h1>Привет, мир!</h1>  
  
<!-- Используем пропсы и  
другие переменные из  
скрипта компонента: -->  
<p>{title}</p>  
  
<!-- Осуществляем  
отложенный рендеринг  
компонента с резервным  
контентом для загрузки: -->
```

```
<Avatar server:defer>
  <svg slot="fallback"
class="generic-avatar"
transition:name="avatar">..
.</svg>
</Avatar>
```

```
<!-- Включаем другие
компоненты
пользовательского
интерфейса с помощью
директивы `client`: для
гидратации: -->
<ReactPokemonComponent
client:visible />
```

```
<!-- Комбинируем HTML с
выражениями JavaScript, как
в JSX: -->
<ul>
```

```
{myFavoritePokemon.map((dat
a) => <li>{data.name}
</li>))}
```

```
</ul>
```

```
<!-- Используем директиву  
шаблона для создания имён  
классов из нескольких строк  
или даже объектов! -->
```

```
<p class:list={["add",  
"dynamic", {classNames:  
true}]} />
```

Компонентный дизайн

Компоненты спроектированы так, чтобы их можно было повторно использовать и компоновать. Вы можете использовать компоненты внутри других компонентов для создания более продвинутого пользовательского интерфейса.

Например, компонент **Button** можно использовать для создания компонента **ButtonGroup**:

```
---  
import Button from  
'./Button.astro';  
---  
<div>  
  <Button title="Кнопка 1"  
>  
  <Button title="Кнопка 2"  
>  
  <Button title="Кнопка 3"  
>  
</div>
```

Слоты

`<slot />` это заполнитель для внешнего HTML-содержимого, позволяющий вставлять дочерние элементы из других файлов в шаблон компонента.

По умолчанию все дочерние элементы, переданные компоненту, будут отображаться в его **<slot />**.

Заметка

В отличие от пропсов, которые являются атрибутами, передаваемыми компоненту **Astro**, доступными для использования во всем вашем компоненте с помощью **Astro.props**, слоты отображают дочерние элементы **HTML** там, где они написаны.

- `src/components/Wrapper.astro`

```
---  
import Header from  
'./Header.astro';  
import Logo from  
'./Logo.astro';  
import Footer from  
'./Footer.astro';
```



```

const { title } =
  Astro.props
---
<div id="content-wrapper">
  <Header />
  <Logo />
  <h1>{title}</h1>
  <slot />  <!-- дочерний
элемент будет здесь -->
  <Footer />
</div>

```

- src/pages/fred.astro

```

---
import Wrapper from
  '../components/Wrapper.astro';
---
<Wrapper title="Fred's
Page">

```

```
<h2>Всё о Фреде</h2>  
<p>Вот немного информации  
о Фреде.</p>  
</Wrapper>
```

Этот шаблон является основой компонента макета **Astro**: целая страница **HTML**-контента может быть «обёрнута» тегами и передана компоненту для отрисовки внутри определённых в нем общих элементов страницы.

Именованные слоты

Компонент **Astro** также может иметь именованные слоты. Это позволяет передавать в местоположение слота только элементы **HTML** с соответствующим именем слота.

Слоты именуются с помощью атрибута ***name***:

- src/components/Wrapper.astro

```
---
import Header from
'./Header.astro';
import Logo from
'./Logo.astro';
import Footer from
'./Footer.astro';

const { title } =
  Astro.props
---
<div id="content-wrapper">
  <Header />
  <!-- дочерние элементы с
атрибутом `slot="after-
header"` будут находиться
здесь -->
  <slot name="after-
```

```
header"/>
  <Logo />
  <h1>{title}</h1>
  <!-- дочерние элементы
без `slot`, или с
`slot="default"` будут
находиться здесь -->
  <slot />
  <Footer />
  <!-- дочерние элементы с
атрибутом `slot="after-
footer"` будут находиться
здесь -->
  <slot name="after-
footer"/>
</div>
```

Чтобы внедрить **HTML**-контент в определённый слот, используйте атрибут **slot** в любом дочернем элементе, чтобы указать имя слота. Все остальные дочерние элементы компонента будут

вставлены в стандартный (безымянный) `<slot />`.

- `src/pages/fred.astro`

```
---
import Wrapper from
'../components/Wrapper.astro';
---
<Wrapper title="Страница
Фреда">
  
  <h2>Всё о Фреде</h2>
  <p>Вот немного информации
о Фреде.</p>
  <p slot="after-
footer">Авторское право
2022</p>
</Wrapper>
```

-
- *** Совет Используйте атрибут **`slot="my-slot"`** в дочернем элементе, который вы хотите передать в соответствующий заполнитель **`<slot name="my-slot" />`** в вашем компоненте.

Чтобы передать несколько HTML-элементов в заполнитель **`<slot/>`** компонента без обёртывающего **`<div>`**, используйте атрибут **`slot=""`** на компоненте **`<Fragment/>`**

- `src/components/CustomTable.astro`

```
---  
// Создание  
пользовательской таблицы с  
именованными заполнителями  
для заголовка и тела  
контента
```

```

---
<table class="bg-white">
  <thead class="sticky top-0 bg-white"><slot
name="header" /></thead>
  <tbody class=" [&_tr:nth-child(odd)]:bg-gray-100">
<slot name="body" />
</tbody>
</table>

```

Вставьте несколько строк и столбцов **HTML**-контента с помощью атрибута ***slot=""***, чтобы указать содержимое ***"header"*** и ***"body"***. Отдельные элементы **HTML** также можно стилизовать:

- `src/components/StockTable.astro`

```

---
import CustomTable from

```

```

'./CustomTable.astro';
---
<CustomTable>
  <Fragment slot="header">
<!-- передаём заголовок
таблицы -->
    <tr><th>Название
продукта</th><th>Единицы на
складе</th></tr>
  </Fragment>

  <Fragment slot="body">
<!-- передаём тело таблицы
-->
    <tr><td>Шлёпанцы</td>
<td>64</td></tr>
    <tr><td>Ботинки</td>
<td>32</td></tr>
    <tr><td>Кроссовки</td>
<td class="text-red-
500">0</td></tr>
  </Fragment>
</CustomTable>

```


Обратите внимание, что именованные слоты должны быть непосредственными дочерними элементами компонента. Вы не можете передавать именованные слоты через вложенные элементы.

- *** Совет

Именованные слоты также можно передавать в [компоненты UI фреймворков!](#)

- *** Заметка

Невозможно динамически генерировать имя слота **Astro**, например, внутри функции **map**. Если эта функция необходима в компонентах UI-фреймворка, лучше всего генерировать эти динамические слоты внутри самого фреймворка.

Резервный контент для слотов

Слоты также могут отображать резервный контент. Когда в слот не передаются соответствующие дочерние элементы, элемент **<slot />** отобразит свои собственные дочерние элементы-заполнители.

- `src/components/Wrapper.astro`

```
---
import Header from
'./Header.astro';
import Logo from
'./Logo.astro';
import Footer from
'./Footer.astro';

const { title } =
Astro.props
---
```

```
<div id="content-wrapper">
  <Header />
  <Logo />
  <h1>{title}</h1>
  <slot>
    <p>Это резервное
содержимое, если в слот не
передан дочерний
элемент</p>
  </slot>
  <Footer />
</div>
```

Резервный контент будет отображаться только в том случае, если нет соответствующих элементов с атрибутом ***slot="name"***, передаваемых в именованный слот.

Astro передаст пустой слот, если элемент слота существует, но не содержит контента для передачи. Резервный

контент нельзя использовать в качестве значения по умолчанию, когда передаётся пустой слот. Резервный контент отображается только тогда, когда элемент слота не найден.

- как по мне почти бессмысленно лучше уж рухнет если что... разве что если отображать и обрабатывать с базы данных... и вывести надпись данных нет но тут астро как по мне проигрывает next.js

Перенос слотов

Слоты можно передавать другим компонентам. Например, при создании вложенных макетов:

- `src/layouts/BaseLayout.astro`



```
---  
---  
<html lang="en">  
  <head>  
    <meta charset="utf-8"  
  />  
    <link rel="icon"  
type="image/svg+xml"  
href="/favicon.svg" />  
    <meta name="viewport"  
content="width=device-  
width" />  
    <meta name="generator"  
content={Astro.generator}  
  />  
    <slot name="head"/>  
  </head>  
  <body>  
    <slot />  
  </body>  
</html>
```

- src/layouts/HomeLayout.astro

```
---  
import BaseLayout from  
"./BaseLayout.astro";  
---  
<BaseLayout>  
  <slot name="head"  
slot="head"/>  
  <slot />  
</BaseLayout>
```

- Результат

```
<html lang="en">  
  <head>  
    <meta charset="utf-8"  
/>  
    <link rel="icon"  
type="image/svg+xml"  
href="/favicon.svg" />
```

```
<meta name="viewport"
content="width=device-
width" />
<meta name="generator"
content="Astro vX.Y.Z" />

<!-- Слот head пуст -->

</head>
<body>
  <!-- Слот по умолчанию
пуст -->
</body>
</html>
```

- *** Моя Заметка Может быть очень полезно для SEO
- *** Заметка

Именованные слоты могут быть переданы другому компоненту,

используя атрибуты ***name*** и ***slot*** в теге **`<slot />`**

Теперь содержимое стандартного (безымянного) слота и слота head, переданных в HomeLayout, будет перенесено в родительский компонент BaseLayout.

- src/pages/index.astro

```
---
import HomeLayout from
"../layouts/HomeLayout.astro";
---
<HomeLayout>
  <title
slot="head">Astro</title>
  <h1>Astro</h1>
</HomeLayout>
```


- Результат

```
<html lang="en">
  <head>
    <meta charset="utf-8"
  />
    <link rel="icon"
type="image/svg+xml"
href="/favicon.svg" />
    <meta name="viewport"
content="width=device-
width" />
    <meta name="generator"
content="Astro vX.Y.Z" />

    <!-- Title "Astro"
попадает сюда благодаря
перенаправлению слотов -->
    <title>Astro</title>

  </head>
  <body>
    <!-- Заголовок "Astro"
```

попадает сюда как основной контент -->

```
<h1>Astro</h1>
</body>
</html>
```

HTML-компоненты

Astro поддерживает импорт и использование *.html* файлов в качестве компонентов или размещение этих файлов в подкаталоге *src/pages/* в качестве страниц. Возможно, вам захочется использовать **HTML**-компоненты, если вы повторно используете код с существующего сайта, созданного без фреймворка, или если вы хотите убедиться, что ваш компонент не имеет динамических функций.

HTML-компоненты должны содержать только валидный **HTML**, поэтому они лишены ключевых возможностей компонентов **Astro**:

Они не поддерживают метаданные, импорт на стороне сервера или динамические выражения. Любые теги **<script>** остаются неупакованными, и обрабатываются, как если бы у них был атрибут **is:inline**. Они могут только ссылаться на ресурсы, которые находятся в папке **public/**.

- *** Заметка

В **HTML**-компоненте элемент **<slot />** будет работать так же, как и в компоненте **Astro**. Чтобы использовать **HTML Web Component Slot**, добавьте атрибут **is:inline** к вашему элементу **<slot>**.

Дополнительно (глубже в кроличью нору)

Fragments

Astro поддерживает `<> </>` нотацию и предоставляет встроенный компонент. Этот компонент может быть полезен для предотвращения использования элементов-обёрток при добавлении `set:*directives` для внедрения **HTML**-строки.

В следующем примере текст абзаца визуализируется с использованием компонента:

```
---  
const htmlString = '<p>Raw  
HTML content</p>';  
---
```

```
<Fragment set:html=  
{htmlString} />
```

set:directives

Справочник директив шаблонов

Директивы шаблона — это особый вид **HTML**-атрибута, доступный внутри любого шаблона компонента **Astro** (**.astro**), и некоторые из них также могут использоваться в **.mdx**.

Директивы шаблона используются для управления поведением элемента или компонента. Директива шаблона может включать какую-либо функцию компилятора, упрощающую вашу работу (например, использование **class:list** вместо **class**). Или директива может указывать компилятору **Astro** на

необходимость выполнения каких-либо особых действий с этим компонентом (например, [гидратации](#) с помощью **client:load**).

В этой части описываются все директивы шаблонов, доступные вам в **Astro**, и принцип их работы.

Правила

Чтобы шаблонная директива была действительной, она должна:

Включите двоеточие :в его имя, используя форму **X:Y**(например: **client:load**). Быть видимым для компилятора (например: *****<X {...attr}>*****не будет работать, если **attr** содержит директиву). Некоторые директивы шаблона (но не все) могут принимать пользовательское значение:

`<X client:load />`(не имеет значения) **`<X class:list={{'some-css-class'}} />`**

(принимает массив) Директива шаблона никогда не включается напрямую в конечный **HTML**-вывод компонента.

Общие директивы

`class:list`

`class:list={...}` Принимает массив значений класса и преобразует их в строку класса. Работает на основе популярной вспомогательной библиотеки [clsx от @lukeed](#).

`class:list` принимает массив из нескольких возможных типов значений:

`string`: Добавлено к элементу **`class`** **`Object`**: Все ключи истины добавляются к

элементу class **Array**: сплюснутый **false**, **null**, или **undefined**: пропущено

```
<!-- This -->
<span class:list={[ 'hello
goodbye', { world: true },
[ 'friend' ] ]} />
<!-- Becomes -->
<span class="hello goodbye
world friend"></span>
```

set:html

set:html={string} внедряет **HTML**-строку в элемент, аналогично настройке ***el.innerHTML***.

Astro не экранирует значение автоматически! Убедитесь, что вы доверяете этому значению, или

экранировали его вручную перед передачей в шаблон. Если вы забудете сделать это, вы рискуете стать жертвой межсайтового скриптинга (XSS).

```
---
const rawHTMLString =
  "Hello
  <strong>World</strong>"
---
<h1>{rawHTMLString}</h1>
  <!-- Output: <h1>Hello
  &lt;strong&gt;World&lt;/strong&gt;</h1> -->
  <h1 set:html=
  {rawHTMLString} />
  <!-- Output: <h1>Hello
  <strong>World</strong></h1>
  -->
```

Вы также можете использовать **set:html** ,
****<Fragment>**** чтобы избежать
добавления ненужного элемента-обёртки.
Это может быть особенно полезно при
получении **HTML** из **CMS**.

```
---  
const cmsContent = await  
fetchHTMLFromMyCMS();  
---  
<Fragment set:html=  
{cmsContent}>
```

****set:html={Promise<string>}**** вставляет
HTML-строку в элемент, заключенный в
Promise.

Это можно использовать для внедрения
HTML-кода, хранящегося снаружи,

например, в базе данных.

```
---  
import api from  
'../db/api.js';  
---  
<article set:html=  
{api.getArticle(Astro.props  
.id)}></article>
```

****set:html=**
{Promise<Response>} внедряет ответ в
элемент.

Это особенно полезно при использовании **fetch()**. Например, при извлечении старых записей из предыдущего генератора статических сайтов.

```
<article set:html=
{fetch('http://example/old-
posts/making-soup.html')}}>
</article>
```

set:html Может использоваться с любым тегом и не обязательно включать HTML. Например, используйте с **JSON.stringify()** с **<script>** тегом, чтобы добавить схему **JSON-LD** на страницу.

```
<script
type="application/ld+json"
set:html={JSON.stringify({
  "@context":
  "https://schema.org/",
  "@type": "Person",
  name: "Houston",
  hasOccupation: {
    "@type": "Occupation",
```

```
        name: "Astronaut"  
      }  
    }})/>
```

set:text

set:text={string} Вставляет текстовую строку в элемент, аналогично настройке **el.innerText**. В отличие от **set:html**, string передаваемое значение автоматически экранируется **Astro**.

Это эквивалентно простой передаче переменной напрямую в выражение шаблона (например: **<div>{someText}</div>**), поэтому эта директива обычно не используется.

client:directives

Эти директивы управляют тем, как [компоненты UI Framework](#) размещаются на странице.

По умолчанию [компонент UI Framework](#) не обрабатывается на клиенте. Если ***client:**** директива не указана, его **HTML**-код отображается на странице без **JavaScript**.

Директиву **client** можно использовать только в [компоненте UI-фреймворка](#), который напрямую импортируется в **.astro** компонент. Директивы **Hydration** не поддерживаются при использовании [динамических тегов](#) и [пользовательских компонентов, передаваемых через components свойство](#) .

client:load

- Приоритет: Высокий

- Полезно для: мгновенно видимых элементов пользовательского интерфейса, которые должны стать интерактивными как можно скорее. Загружайте и гидратируйте компонент **JavaScript** немедленно при загрузке страницы.

```
<BuyButton client:load />
```

client:idle

- Приоритет: Средний
- Полезно для: низкоприоритетных элементов пользовательского интерфейса, которые не требуют немедленного взаимодействия. Загрузите и обновите **JavaScript**-компонент после завершения

первоначальной загрузки страницы и `requestIdleCallback` срабатывания события. Если ваш браузер не поддерживает, будет использоваться событие `requestIdleCallback` документа `.load`

```
<ShowHideButton client:idle  
/>
```

timeout

Добавлено в: astro@4.15.0

Максимальное время ожидания (в **миллисекундах**) перед загрузкой компонента, даже если начальная загрузка страницы еще не завершена.

Это позволяет передавать значение параметра `timeout` из `requestIdleCallback()` спецификации . Это означает, что вы можете отложить гидратацию для элементов пользовательского интерфейса с более низким приоритетом, обеспечивая больший контроль и гарантируя интерактивность элемента в течение заданного периода времени.

```
<ShowHideButton  
  client:idle={{timeout:  
    500}} />
```

client:visible

- Приоритет: низкий
- Полезно для: низкоприоритетных элементов пользовательского

интерфейса, которые либо находятся далеко внизу страницы («ниже сгиба»), либо требуют настолько много ресурсов для загрузки, что вы бы предпочли не загружать их вообще, если пользователь никогда не увидит этот элемент. Загружайте и гидратируйте **JavaScript**-код компонента после того, как он попадает в область просмотра пользователя. **IntersectionObserver** Для отслеживания видимости используется внутренний механизм.

```
<HeavyImageCarousel  
  client:visible />
```

client:visible={{rootMargin}}

Добавлено в: astro@4.1.0

При необходимости значение **rootMargin** может быть передано базовому **IntersectionObserver**. Если **rootMargin** указано, **JavaScript**-компонент будет гидратироваться, когда заданное поле (в пикселях) вокруг компонента попадает в область просмотра, а не сам компонент.

```
<HeavyImageCarousel  
  client:visible=  
  {{rootMargin: "200px"}} />
```

Указание **rootMargin** значения может сократить сдвиги макета (CLS), дать компоненту больше времени для загрузки при более медленном интернет-соединении и сделать компоненты интерактивными быстрее, повысив

стабильность и скорость реагирования страницы.

client:media

- Приоритет: низкий
- Полезно для: переключателей боковой панели и других элементов, которые могут быть видны только на экранах определенных размеров.
client:media={string} загружает и обновляет компонент **JavaScript** после выполнения определенного медиа-запроса **CSS**.

Примечание

Если компонент уже скрыт и показан с помощью медиа-запроса в вашем **CSS**, то может быть проще просто использовать

его **client:visible** и не передавать тот же медиа-запрос в директиву.

```
<SidebarToggle  
  client:media="(max-width:  
  50em)" />
```

client:only

client:only={string} Пропускает рендеринг **HTML** на сервере и отображает его только на клиенте. Он действует аналогично:

client:load загружает, отображает и обновляет компонент сразу после загрузки страницы.

Необходимо передать корректную платформу компонента в качестве значения! Поскольку **Astro** не запускает компонент во время сборки на сервере,

Astro не знает, какую платформу использует ваш компонент, пока вы не укажете её явно.

```
<SomeReactComponent  
  client:only="react" />  
<SomePreactComponent  
  client:only="preact" />  
<SomeSvelteComponent  
  client:only="svelte" />  
<SomeVueComponent  
  client:only="vue" />  
<SomeSolidComponent  
  client:only="solid-js" />
```

Отображение загружаемого содержимого

Для компонентов, которые отображаются только на клиенте, также можно

отображать резервный контент во время загрузки. Используйте ***slot="fallback"*** этот параметр для любого дочернего элемента, чтобы создать контент, который будет отображаться только до тех пор, пока ваш клиентский компонент не станет доступен:

```
<ClientComponent
  client:only="vue">
  <div
    slot="fallback">Loading</div>
  </ClientComponent>
```

Индивидуальные клиентские директивы

Начиная с версии Astro 2.6.0 интеграции также позволяют добавлять

пользовательские ***client:**** директивы для изменения того, как и когда следует гидратировать компоненты.

Посетите часть [addClientDirectiveAPI](#) , чтобы узнать больше о создании пользовательской клиентской директивы.

Директивы сервера

Эти директивы управляют отображением компонентов серверного острова.

server:defer

Директива ***server:defer*** преобразует компонент в серверный остров, заставляя его отображаться по требованию, вне области отображения остальной части страницы.

Подробнее об использовании компонентов серверного острова [см.](#)

```
<Avatar server:defer />
```

Директивы по скрипту и стилю

Эти директивы можно использовать только в **HTML** `<script>` и `<style>` тегах для управления тем, как клиентский **JavaScript** и **CSS** обрабатываются на странице.

is:global

По умолчанию **Astro** автоматически применяет `<style>` **CSS**-правила к компоненту. Вы можете отключить это поведение с помощью ***is:global*** директивы.

is:global При включении компонента содержимое тега `<style>` применяется глобально на странице. Это отключает систему **CSS**-областей действия **Astro**. Это эквивалентно заключению всех селекторов в `<style>` тег с ***:global()***.

Вы можете объединить `<style>` и `<style is:global>` в одном компоненте, чтобы создать некоторые глобальные правила стиля, сохраняя при этом область действия большей части **CSS** вашего компонента.

Более подробную информацию о работе глобальных стилей см . на странице [«Стилизация и CSS»](#) .

```
<style is:global>
  body a { color: red; }
</style>
```

is:inline

По умолчанию **Astro** обрабатывает, оптимизирует и объединяет все теги `<script>` и `<style>` которые видит на странице. Вы можете отключить это поведение с помощью ***is:inline*** директивы.

is:inline Astro указывает оставить тег `<script>` или `<style>` как есть в конечном **HTML**-коде. Содержимое не будет обработано, оптимизировано или упаковано. Это ограничивает некоторые функции **Astro**, такие как импорт пакета **npm** или использование языка компиляции в **CSS**, например, **Sass**.

Директива ***is:inline*** означает, что `<style>` и `<script>` теги:

- Не будет объединено во внешний файл. Это означает, что атрибуты, например `defer`, управляющие загрузкой внешнего файла, не будут иметь никакого эффекта.
- Дедупликация не производится — элемент будет появляться столько раз, сколько он отображается.
- Ссылки `// import` не будут разрешены относительно файла `.@import url().astro`
- Будет отображен в конечном выходном **HTML**-коде именно там, где он был создан.
- Стили будут глобальными и не будут ограничены компонентом.
- *** Осторожность

Эта `is:inline` директива подразумевается всякий раз, когда в теге используется любой атрибут, кроме `.src`. Единственным исключением является использование директивы в теге, которое не подразумевает автоматически.

```
<script>  
<style>define:vars<style>is  
:inline
```

```
<style is:inline>  
  /* inline: relative & npm  
  package imports are not  
  supported. */  
  @import '/assets/some-  
public-styles.css';  
  span { color: green; }  
</style>
```

```
<script is:inline>
  /* inline: relative & npm
  package imports are not
  supported. */
  console.log('I am inlined
  right here in the final
  output HTML.');
```

Посмотрите, как работают [клиентские скрипты](#) в компонентах Astro.

define:vars

define:vars={...} Может передавать серверные переменные из ***frontmatter*** вашего компонента в клиент `<script>` или `<style>` теги. Поддерживаются любые **JSON**-сериализуемые переменные

frontmatter, в том числе **props**

переданные в ваш компонент через

Astro.props. Значения сериализуются с помощью ***JSON.stringify()***.

```
---
const foregroundColor =
"rgb(221 243 228)";
const backgroundColor =
"rgb(24 121 78)";
const message = "Astro is
awesome!";
---
<style define:vars={{
  textColor: foregroundColor,
  backgroundColor }}>
  h1 {
    background-color: var(--
-backgroundColor);
    color: var(--
textColor);
  }
```

```
</style>

<script define:vars={{
  message }}>
  alert(message);
</script>
```

- *** Осторожность

Использование ***define:vars*** тега `<script>` подразумевает ***is:inline*** директиву , что означает, что ваши скрипты не будут объединены, а будут встроены непосредственно в **HTML**.

Это связано с тем, что когда **Astro** объединяет скрипт, он включает и запускает его один раз, даже если вы включаете компонент, содержащий скрипт, несколько раз на одной странице. ***define:vars*** требует повторного запуска

скрипта с каждым набором значений, поэтому вместо этого **Astro** создает встроенный скрипт.

Для скриптов попробуйте передавать переменные в скрипты вручную .

Расширенные директивы

is:raw

is:raw указывает компилятору **Astro** обрабатывать все дочерние элементы этого элемента как текст. Это означает, что весь специальный синтаксис шаблонов **Astro** будет игнорироваться внутри этого компонента.

Например, если у вас есть пользовательский компонент **Katex**, который преобразует текст в **HTML**, вы

можете попросить пользователей сделать следующее:

```
---  
import Katex from  
'../components/Katex.astro'  
;  
---  
<Katex is:raw>Some  
conflicting {syntax}  
here</Katex>
```

framework-components

Фронтенд-фреймворки

Создайте свой **Astro** сайт, не жертвуя любимым фреймворком компонентов. Создавайте астрологические острова , используя **UI-фреймворки** на ваш выбор.

Официальная интеграция фронтенд-фреймворков **Astro** поддерживает множество популярных фреймворков

В [каталоге](#) интеграций вы найдете еще больше интеграций с фреймворками, поддерживаемыми сообществом (например, Angular, Qwik, Elm).

UI-фреймворки:

[@astrojs/ alpinejs](#)

[@astrojs/ preact](#)

[@astrojs/ react](#)

[@astrojs/ solid-js](#)

[@astrojs/ svelte](#)

[@astrojs/ vue](#)

- можно установить и настроить одну или несколько интеграций.

Подробную информацию об установке и настройке интеграций **Astro** см. в [Руководстве по интеграциям](#).

Использование компонентов фреймворка

Используйте компоненты вашего **JavaScript**-фреймворка на страницах, макетах и в компонентах **Astro** так же, как и компоненты **Astro**! Все ваши компоненты могут располагаться вместе */src/components* или быть организованы любым удобным для вас способом.

Чтобы использовать компонент фреймворка, импортируйте его по относительному пути в скрипте вашего компонента **Astro**. Затем используйте

компонент вместе с другими компонентами, **HTML**-элементами и выражениями, подобными **JSX**, в шаблоне компонента.

- `src/pages/static-components.astro`

```
---
import MyReactComponent
from
  '../components/MyReactComponent.jsx';
---
<html>
  <body>
    <h1>Use React
components directly in
Astro!</h1>
    <MyReactComponent />
  </body>
</html>
```

По умолчанию компоненты вашего фреймворка будут отображаться на сервере только в виде статического **HTML**. Это полезно для создания шаблонов неинтерактивных компонентов и позволяет избежать отправки лишнего **JavaScript** на клиентскую сторону.

Увлажняющие интерактивные компоненты

Компонент фреймворка можно сделать интерактивным (гидратированным) с помощью ***client:**** директивы. Это атрибуты компонента, которые определяют, когда **JavaScript**-код компонента должен быть отправлен в браузер.

При использовании всех клиентских директив, кроме `<script>` ***client:only***,

ваш компонент сначала будет отображаться на сервере для генерации статического **HTML. JavaScript** компонента будет отправлен в браузер в соответствии с выбранной вами директивой. Затем компонент будет инициализирован и станет интерактивным.

- `src/pages/interactive-components.astro`

```
---  
// Example: hydrating  
framework components in the  
browser.  
import InteractiveButton  
from  
'../components/InteractiveB  
utton.jsx';  
import InteractiveCounter  
from
```

```
'../components/InteractiveCounter.jsx';
import InteractiveModal
from
'../components/InteractiveModal.svelte';
---
<!-- This component's JS
will begin importing when
the page loads -->
<InteractiveButton
client:load />

<!-- This component's JS
will not be sent to the
client until
the user scrolls down and
the component is visible on
the page -->
<InteractiveCounter
client:visible />

<!-- This component won't
render on the server, but
```



```
will render on the client  
when the page loads -->  
<InteractiveModal  
  client:only="svelte" />
```

Необходимый для рендеринга компонента **JavaScript**-фреймворк (**React**, **Svelte** и т. д.) будет отправлен в браузер вместе с собственным **JavaScript**-кодом компонента. Если два или более компонентов на странице используют один и тот же фреймворк, фреймворк будет отправлен только один раз.

- ***Доступность

Большинство специфических для фреймворков шаблонов обеспечения доступности должны работать одинаково при использовании этих компонентов в **Astro**. Обязательно выберите директиву

клиента, которая обеспечит правильную загрузку и выполнение всего **JavaScript**-кода, связанного с доступностью, в нужное время!

Доступные рекомендации по поддержанию баланса загрузки

Для компонентов пользовательского интерфейса доступно несколько директив гидратации: **client:load**, **client:idle**, **client:visible**, **client:media={QUERY}** и **client:only={FRAMEWORK}**.

Полное описание этих рекомендаций по гидратации и их применению вы найдете в [справочной информации](#).

Смешивание фреймворков

В одном и том же компоненте Astro можно импортировать и отображать

компоненты из разных фреймворков.

- src/pages/mixing-frameworks.astro

```
---  
// Example: Mixing multiple  
framework components on the  
same page.  
import MyReactComponent  
from  
'../components/MyReactCompo  
nent.jsx';  
import MySvelteComponent  
from  
'../components/MySvelteComp  
onent.svelte';  
import MyVueComponent from  
'../components/MyVueCompone  
nt.vue';  
---  
<div>  
  <MySvelteComponent />  
  <MyReactComponent />
```

```
<MyVueComponent />  
</div>
```

Astro распознает и отобразит ваш компонент на основе расширения его файла. Для различения фреймворков, использующих одно и то же расширение файла, требуется дополнительная настройка при рендеринге нескольких **JSX**-фреймворков (например, **React** и **Preact**).

- ***Осторожность

Только компоненты **Astro .astro** могут содержать компоненты из нескольких фреймворков.

Передача свойств компонентам фреймворка

Вы можете передавать свойства из компонентов **Astro** в компоненты фреймворка:

- `src/pages/frameworks-props.astro`

```
---
import TodoList from
'../components/TodoList.jsx
';
import Counter from
'../components/Counter.svelte';
---
<div>
  <TodoList initialTodos=
[["learn Astro", "review
PRs"]] />
  <Counter startingCount=
{1} />
</div>
```

Свойства, передаваемые компонентам интерактивной среды разработки с помощью ***client:**** директивы, должны быть **сериализованы**: преобразованы в формат, подходящий для передачи по сети или хранения. Однако **Astro** сериализует не все типы структур данных. Поэтому существуют некоторые ограничения на то, что можно передавать в качестве свойств компонентам, прошедшим гидратацию.

Поддерживаются следующие типы свойств: простой объект number, и string
Array Map Set Reg Exp Date BigInt URL
Uint8Array Uint16Array Uint32Array Infinity

Неподдерживаемые структуры данных, передаваемые компонентам, такие как функции, могут использоваться только во время серверного рендеринга

компонента и не могут использоваться для обеспечения интерактивности. Например, передача функций в гидратированные компоненты не поддерживается, поскольку **Astro** не может передавать функции с сервера таким образом, чтобы они были исполняемыми на клиенте.

- [подробнее](#)

Передача дочерних элементов компонентам фреймворка.

Внутри компонента Astro можно передавать дочерние элементы компонентам фреймворка. Каждый фреймворк имеет свои собственные шаблоны обращения к этим дочерним элементам: **React**, **Preact** и **Solid** используют специальное свойство с

именем `<pre>` children, а **Svelte** и **Vue** используют `<slot />` элемент `<pre>`.

- src/pages/component-children.astro

```
---
import MyReactSidebar from
'../components/MyReactSideb
ar.jsx';
---
<MyReactSidebar>
  <p>Here is a sidebar with
some text and a button.</p>
</MyReactSidebar>
```

Кроме того, вы можете использовать **именованные слоты** для группировки определенных дочерних элементов.

Для **React**, **Preact** и **Solid** эти слоты будут преобразованы в свойство верхнего

уровня. Названия слотов, использующие, **kebab-case** будут преобразованы в **camelCase**.

- `src/pages/named-slots.astro`

```
---
import MySidebar from
'../components/MySidebar.js
x';
---
<MySidebar>
  <h2
slot="title">Menu</h2>
  <p>Here is a sidebar with
some text and a button.</p>
  <ul slot="social-links">
    <li><a
href="https://twitter.com/a
strodotbuild">Twitter</a>
</li>
    <li><a
href="https://github.com/wi
```

```
thastro">GitHub</a></li>
</ul>
</MySidebar>
```

- src/components/MySidebar.jsx

```
export default function
MySidebar(props) {
  return (
    <aside>
      <header>{props.title}
    </header>
      <main>
        {props.children}</main>
      <footer>
        {props.socialLinks}
      </footer>
    </aside>
  )
}
```

В **Svelte** и **Vue** эти слоты можно указывать с помощью **<slot>** элемента с *name* атрибутом. Имена слотов, использующие атрибут, **kebab-case** будут сохранены.

- src/components/MySidebar.svelte

```
<aside>
  <header><slot
name="title" /></header>
  <main><slot /></main>
  <footer><slot
name="social-links" />
</footer>
</aside>
```

Компоненты фреймворка вложенности

Внутри файла **Astro** дочерние компоненты фреймворка также могут быть «гидратированными» компонентами.

Это означает, что вы можете рекурсивно вкладывать компоненты из любого из этих фреймворков.

- src/pages/nested-components.astro

```
---
import MyReactSidebar from
'../components/MyReactSideb
ar.jsx';
import MyReactButton from
'../components/MyReactButto
n.jsx';
import MySvelteButton from
'../components/MySvelteButt
on.svelte';
---
<MyReactSidebar>
  <p>Here is a sidebar with
some text and a button.</p>
  <div slot="actions">
    <MyReactButton
client:idle />
```

```
<MySvelteButton  
  client:idle />  
</div>  
</MyReactSidebar>
```

- ***Осторожно Помните: сами файлы компонентов фреймворка (например .jsx, .svelte) не могут использоваться для смешивания нескольких фреймворков.

Это позволяет создавать целые «приложения» в предпочитаемом вами **JavaScript**-фреймворке и отображать их через родительский компонент на странице **Astro**.

- Примечание

Компоненты **Astro** всегда отображаются в статическом **HTML**, даже если они включают в себя компоненты

фреймворка, которые уже заполнены. Это означает, что вы можете передавать только те свойства, которые не выполняют рендеринг **HTML**. Передача свойств рендеринга **React** компонентам фреймворка из компонента **Astro** не сработает, поскольку компоненты **Astro** не могут обеспечить поведение во время выполнения на стороне клиента, которое требуется для этого шаблона. Вместо этого используйте именованные слоты.

Можно ли использовать компоненты Astro внутри компонентов моего фреймворка?

Любой компонент **UI**-фреймворка становится «островом» этого фреймворка. Эти компоненты должны быть написаны полностью как допустимый код для данного фреймворка, используя только

его собственные импорты и пакеты. Вы не можете импортировать **.astro** компоненты в компонент **UI**-фреймворка (например **.jsx**, или **.svelte**).

Однако вы можете использовать паттерн **Astro <slot />** для передачи статического контента, сгенерированного компонентами **Astro**, в качестве дочерних элементов компонентам вашей платформы внутри **.astro** компонента .

- src/pages/astro-children.astro

```
---  
import MyReactComponent  
from  
'../components/MyReactComponent.jsx';  
import MyAstroComponent  
from  
'../components/MyAstroComponent.jsx';
```

```
nent.astro';  
---  
<MyReactComponent>  
  <MyAstroComponent  
    slot="name" />  
</MyReactComponent>
```

Можно ли hydrate компоненты Astro?

Если вы попытаетесь инициализировать компонент **Astro** с помощью *client:модификатора*, вы получите ошибку.

Компоненты **Astro** — это компоненты, использующие только **HTML**-шаблоны и не имеющие клиентской среды выполнения. Однако вы можете использовать **<script>** тег в шаблоне компонента **Astro** для отправки **JavaScript** в браузер, который будет

выполняться в глобальной области видимости.

Сериализация

Процесс, в ходе которого объект или структура данных преобразуются в формат, подходящий для передачи по сети или хранения (например, в виде буфера массива или файлового формата).

В **JavaScript** , например, вы можете сериализовать объект в строку JSON , вызвав соответствующую функцию **JSON.stringify()**

Значения **CSS** сериализуются путем вызова функции **CSSStyleDeclaration.getPropertyValue()**.

Неподдерживаемые структуры данных, передаваемые

компонентам, такие как функции

Это ограничение касается того, как **Astro** обрабатывает данные при переходе с сервера в браузер, и оно очень важно для понимания концепции гидратации.

Давайте разберем эту сложную строчку простым, человеческим языком.

Суть проблемы: Граница между сервером и клиентом

Вся проблема заключается в том, что ваш сервер (где **Astro** генерирует **HTML**) и браузер пользователя (клиент, где **JavaScript** делает страницу интерактивной) — это два разных мира с разными правилами.

 Мир 1: Сервер (Строительная площадка) На сервере у вас есть полный

контроль. Вы можете использовать любые функции **JavaScript**, логику, подключать базы данных. Astro использует этот мир для создания статического **HTML**-кода.

🌐 Мир 2: Клиент (Браузер пользователя) В браузере работает только тот код, который вы туда отправили. Чтобы компонент стал интерактивным (гидратировался), **Astro** должен отправить его данные и логику в браузер.

Почему функции "не переезжают" из мира 1 в мир 2?

Когда **Astro** пытается отправить данные с сервера в браузер, он использует процесс, похожий на упаковку багажа. Данные должны быть преобразованы в универсальный, читаемый формат,

который называется **JSON**. **JSON** — это просто текст, который может понять и сервер, и браузер.

JSON умеет упаковывать простые вещи: Строки ("Привет") Числа (42) Булевы значения (true/false) Массивы и простые объекты.

НО **JSON** НЕ УМЕЕТ ПАКОВАТЬ ФУНКЦИИ.

Функция — это набор инструкций (кусочек кода), который имеет смысл только в том окружении, где он был написан.

Объяснение фразы "человеческим языком"

«...передача функций в гидратированные компоненты не поддерживается, поскольку **Astro** не может передавать

функции с сервера таким образом, чтобы они были исполняемыми на клиенте.»

Это значит:

Вы не можете написать функцию на сервере и ожидать, что она магическим образом начнет работать в браузере пользователя, когда компонент "оживет" (гидратируется). **Astro** пытается перевести все данные компонента в текстовый формат (**JSON**) для отправки в браузер. Функции при этом просто выбрасываются или превращаются в **undefined**, потому что их нельзя сериализовать (упаковать).

Аналогия с переездом:

Представьте, что вы переезжаете из Казахстана в Германию. Вы можете взять с собой мебель, одежду, книги (простые

данные). Но вы не можете взять с собой казахстанские законы или свою работу в компании и ожидать, что они будут работать в Берлине. Вам нужно устроиться на новую работу и изучить новые законы на месте.

В **Astro**: Мебель/Одежда = Строки, числа, массивы (передаются легко). Функции/Логика сервера = Законы/работа (нельзя передать, их нужно реализовывать заново на стороне клиента, если они вам там нужны). Пример неправильного использования

Вы не сможете сделать так:

```
---  
// ЭТО КОД НА СЕРВЕРЕ  
(Astro Island)  
const handleClick = () => {
```

```
console.log("Клик  
произошел на сервере!");  
}  
---  
<!-- ПЕРЕДАЧА ФУНКЦИИ В  
ГИДРАТИРУЕМЫЙ КОМПОНЕНТ -->  
<MyButton client:visible  
onClick={handleClick} />  
Use code with caution.
```

Когда **MyButton** гидратируется в браузере, функция **handleClick** просто не будет существовать.

! Правильное решение Если вам нужна интерактивность, логика должна быть написана внутри самого клиентского компонента:

```
// ЭТО JSX/TSX ФАЙЛ,  
КОТОРЫЙ РАБОТАЕТ И НА
```

КЛИЕНТЕ И НА СЕРВЕРЕ

```
import { useState } from
'react';

export default function
MyButton() {
  // handleClick определена
внутри компонента и будет
работать в браузере
  const handleClick = () =>
  {
    alert("Клик сработал в
браузере!");
  }

  return (
    <button onClick=
{handleClick}>
      Нажми меня
    </button>
  );
}
```


Astro-syntax

Справочник шаблонных выражений

Синтаксис компонентов **Astro** является расширенным вариантом **HTML**. Он разработан таким образом, чтобы быть понятным любому, кто имеет опыт написания **HTML** или **JSX**, и добавляет поддержку включения компонентов и выражений **JavaScript**.

Выражения, подобные JSX

Вы можете определить локальные переменные **JavaScript** внутри скрипта компонента **frontmatter** между двумя блоками кода (**---**) компонента **Astro**. Затем вы можете внедрить эти переменные в **HTML**-шаблон компонента, используя выражения, похожие на **JSX**!

- Динамический против реактивного

Используя этот подход, вы можете включать динамические значения, которые вычисляются в метаданных. Но после включения эти значения не являются реактивными и никогда не изменятся. Компоненты **Astro** — это шаблоны, которые выполняются только один раз, на этапе рендеринга.

Ниже приведены дополнительные примеры различий между **Astro** и **JSX**.

Переменные

Локальные переменные можно добавлять в **HTML**, используя синтаксис фигурных скобок:

- src/components/Variables.astro

```
---  
const name = "Astro";  
---  
<div>  
  <h1>Hello {name}!</h1>  
<!-- Outputs <h1>Hello  
Astro!</h1> -->  
</div>
```

Динамические атрибуты

Локальные переменные можно использовать в фигурных скобках для передачи значений атрибутов как **HTML**-элементам, так и компонентам:

- src/components/DynamicAttributes.astro

```
---  
const name = "Astro";  
---  
<h1 class={name}>Attribute  
expressions are  
supported</h1>  
  
<MyComponent  
templateLiteralNameAttribut  
e={`MyNameIs${name}`} />
```

- Осторожность

Атрибуты **HTML** будут преобразованы в строки, поэтому передавать функции и объекты элементам **HTML** невозможно. Например, вы не можете назначить обработчик событий элементу **HTML** в компоненте **Astro**:

- dont-do-this.astro

```
---  
function handleClick () {  
    console.log("button  
clicked!");  
}  
---  
<!-- ✖ This doesn't work! ✖  
-->  
<button onClick=  
{handleClick}>Nothing will  
happen when you click me!  
</button>
```

Вместо этого используйте клиентский скрипт для добавления обработчика событий, как это делается в чистом JavaScript:

- do-this-instead.astro

```
---  
---  
<button id="button">Click  
Me</button>  
<script>  
    function handleClick () {  
        console.log("button  
clicked!");  
    }  
  
    document.getElementById("bu  
tton").addEventListener("cl  
ick", handleClick);  
</script>
```

Динамический HTML

Локальные переменные можно использовать в функциях, подобных **JSX**, для создания динамически генерируемых **HTML**-элементов:

- src/components/DynamicHtml.astro

```
---  
const items = ["Dog",  
               "Cat", "Platypus"];  
---  
<ul>  
  {items.map((item) => (  
    <li>{item}</li>  
  ))}  
</ul>
```

Astro может отображать **HTML**-код в зависимости от условий, используя логические операторы **JSX** и тернарные выражения.

- src/components/ConditionalHtml.astro

```
---  
const visible = true;  
---  
{visible && <p>Show me!  
</p>}  
  
{visible ? <p>Show me!</p>  
: <p>Else show me!</p>}
```

Динамические теги

Вы также можете использовать динамические теги, присвоив имя **HTML**-тега переменной или переназначив импорт компонента:

- `src/components/DynamicTags.astro`

```
---  
import MyComponent from
```



```
"/MyComponent.astro";  
const Element = 'div'  
const Component =  
MyComponent;  
---  
<Element>Hello!</Element>  
<!-- renders as <div>Hello!  
</div> -->  
<Component /> <!-- renders  
as <MyComponent /> -->
```

При использовании динамических тегов:

- Названия переменных должны быть написаны с заглавной буквы. Например, используйте **Element**, а не **element**. В противном случае **Astro** попытается отобразить имя вашей переменной как обычный **HTML**-тег.

- Директивы гидратации не поддерживаются. При использовании ***client:директив** гидратации **Astro** необходимо знать, какие компоненты следует включить в состав для использования в продакшене, а динамический шаблон тегов препятствует этому.
- Директива **define:vars** не поддерживается. Если вы не можете обернуть дочерние элементы дополнительным элементом (например, <div>), то вы можете вручную добавить элемент **style={-myVar : \${value}}** к вашему элементу.

Фрагменты

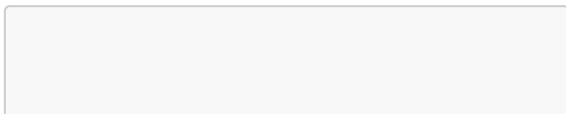
Различия между Astro и JSX

.astro Синтаксис компонентов **Astro** является надмножеством **HTML**. Он был разработан таким образом, чтобы быть понятным любому, кто имеет опыт работы с **HTML** или **JSX**, но между файлами и **JSX** есть несколько ключевых различий .

Атрибуты

В **Astro** для всех **HTML**-атрибутов используется стандартный **kebab-case** формат, а не тот, что **camelCase** применяется в **JSX**. Это работает даже для **<a> class**, который не поддерживается **React**.

- пример **.astro**



```
<div className="box"  
  dataValue="3" />  
<div class="box" data-  
  value="3" />
```

Несколько элементов

В отличие от **JavaScript** или **JSX**, шаблон компонента **Astro** позволяет отображать несколько элементов без необходимости оборачивать всё в один **<div>** или несколько элементов. **<>**

- `src/components/RootElements.astro`

```
---  
// Template with multiple  
elements  
---  
<p>No need to wrap elements
```

```
in a single containing
element.</p>
<p>Astro supports multiple
root elements in a
template.</p>
```

Комментарии

В **Astro** можно использовать стандартные **HTML**-комментарии или комментарии в стиле **JavaScript**.

- пример **.астро**

```
---
---
<!-- HTML comment syntax is
valid in .astro files -->
{/* JS comment syntax is
also valid */}
```

- ***Осторожно

Комментарии в стиле **HTML** будут включены в **DOM** браузера, а комментарии в стиле **JavaScript** будут пропущены. Для сообщений типа **TODO** или других пояснений, предназначенных только для разработки, вы можете использовать комментарии в стиле **JavaScript**.

Вспомогательные утилиты компонентов

Astro.slots

Astro.slots Содержит вспомогательные функции для изменения дочерних элементов компонента **Astro**, расположенных в слотах.

Astro.slots.has()

Тип: **(slotName: string) => boolean**

Проверить наличие контента для конкретного имени слота можно с помощью метода **Astro.slots.has()**.. Это может быть полезно, если вы хотите обернуть содержимое слота, но хотите отображать элементы-обертки только тогда, когда слот используется.

- `src/pages/index.astro`

```
---
---
<slot />

{Astro.slots.has('more') &&
(
  <aside>
    <h2>More</h2>
    <slot name="more" />
```

```
</aside>  
)}
```

Astro.slots.render()

Тип: **(slotName: string, args?: any[]) => Promise**

Вы можете асинхронно отображать содержимое слота в строку **HTML**, используя **Astro.slots.render()**.

```
---  
const html = await  
Astro.slots.render('default  
' );  
---  
<Fragment set:html={html}  
>
```

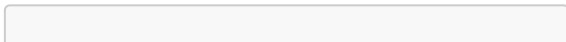

- ***Примечание

Это предназначено для сложных сценариев использования! В большинстве случаев проще отображать содержимое слота с помощью элемента `<slot />` .

Astro.slots.render() Функция может опционально принимать второй аргумент: массив параметров, которые будут переданы дочерним функциям. Это может быть полезно для пользовательских вспомогательных компонентов.

Например, этот `<Shout />` компонент преобразует свое **message** свойство в верхний регистр и передает его в слот по умолчанию:

- `src/components/Shout.astro`



```
---  
const message =  
  Astro.props.message.toUpperCase()  
;  
let html = '';  
if  
  (Astro.slots.has('default'))  
  {  
    html = await  
      Astro.slots.render('default',  
        [message]);  
  }  
---  
<Fragment set:html={html}  
>
```

Функция обратного вызова, переданная в качестве **<Shout />** дочернего элемента, получит **message** параметр, написанный заглавными буквами:

- `src/pages/index.astro`

```
---  
import Shout from  
  "../components/Shout.astro"  
;  
---  
<Shout message="slots!">  
  {(message) => <div>  
    {message}</div>}  
</Shout>  
  
<!-- renders as <div>SLOTS!  
</div> -->
```

Функции обратного вызова можно передавать в именованные слоты внутри обертывающего **HTML**-элемента с **slot** атрибутом. Этот элемент используется только для передачи функции обратного вызова в именованный слот и не будет отображаться на странице.

```
<Shout message="slots!">
  <fragment slot="message">
    {(message) => <div>
{message}</div>}
  </fragment>
</Shout>
```

Для тега-обертки используйте стандартный **HTML**-элемент или любой тег в нижнем регистре (например, **<fragment>** вместо **<Fragment />**), который не будет интерпретироваться как компонент. Не используйте **HTML**- **<slot>** элемент, так как он будет интерпретироваться как слот **Astro**.

Astro.self

Astro.self Это позволяет рекурсивно вызывать компоненты **Astro**. Такое

поведение позволяет отображать компонент **Astro** изнутри самого себя, используя его **<Astro.self>** в шаблоне компонента. Это может помочь при итерации по большим хранилищам данных и вложенным структурам данных.

- NestedList.astro

```
---
const { items } =
  Astro.props;
---
<ul class="nested-list">
  {items.map((item) => (
    <li>
      <!-- If there is a
nested data-structure we
render `<Astro.self>` -->
      <!-- and can pass
props through with the
recursive call -->
```

```

        {Array.isArray(item)
? (
        <Astro.self items=
{item} />
        ) : (
        item
        )}
    </li>
  )}}
</ul>

```

Затем этот компонент можно использовать следующим образом:

```

---
import NestedList from
'./NestedList.astro';
---
<NestedList items={['A',
['B', 'C'], 'D']} />

```

И отобразит HTML следующим образом:

```
<ul class="nested-list">
  <li>A</li>
  <li>
    <ul class="nested-
list">
      <li>B</li>
      <li>C</li>
    </ul>
  </li>
  <li>D</li>
</ul>
```

passing-components-to-mdx-content

@astrojs/mdx

Эта интеграция с **Astro** позволяет использовать компоненты **MDX** и создавать страницы в виде **.mdx** файлов.

Почему именно MDX?

MDX позволяет использовать переменные, выражения **JSX** и компоненты в контенте **Markdown** в **Astro**. Если у вас уже есть контент, созданный в формате **MDX**, эта интеграция позволит вам интегрировать эти файлы в ваш проект **Astro**.

Установка

В **Astro** есть ***astro add*** команда для автоматической настройки официальных интеграций. При желании вы можете установить интеграции вручную .

Выполните одну из следующих команд в новом окне терминала.

```
npx astro add mdx
```


Если у вас возникнут какие-либо проблемы, [сообщите о них на GitHub](#) и попробуйте выполнить описанные ниже шаги по ручной установке.

Ручная установка

Сначала установите @astrojs/mdx пакет:

```
npm install @astrojs/mdx
```

Затем примените интеграцию к вашему ***astro.config.**** файлу, используя следующее ***integrations*** свойство:

- astro.config.mjs

```
import { defineConfig }  
from 'astro/config';  
import mdx from  
'@astrojs/mdx';  
  
export default  
defineConfig({  
  // ...  
  integrations: [mdx()],  
});
```

Интеграция с редактором

Для поддержки редактора в [VS Code](#) установите [официальное расширение MDX](#).

Для других редакторов используйте [языковой сервер MDX](#).

Использование

Чтобы узнать об использовании стандартных функций **MDX**, посетите документацию **MDX** .

MDX в Астро

Добавление интеграции с **MDX** расширяет возможности создания **Markdown**-макетов с помощью переменных, выражений и компонентов **JSX**.

Это также добавляет дополнительные функции к стандартному **MDX**, включая поддержку метаданных в стиле **Markdown**. Это позволяет использовать [большинство встроенных в Astro функций Markdown](#) .

.mdx Файлы должны быть написаны в [синтаксисе MDX](#), а не в синтаксисе, похожем на **HTML**, используемом в **Astro**.

Использование MDX с коллекциями контента

Чтобы включить MDX-файлы в коллекцию контента, убедитесь, что ваш загрузчик коллекций настроен на загрузку контента из **.mdx** файлов:

- `src/content.config.ts`

```
import { defineCollection }
from 'astro:content';
import { glob } from
'astro/loaders';
import { z } from
'astro/zod';

const blog =
defineCollection({
  loader: glob({ pattern:
    "**/*.md,mdx", base:
    "./src/blog" }),
```

```
    schema: z.object({
      title: z.string(),
      description:
z.string(),
      pubDate:
z.coerce.date(),
    })
  });

export const collections =
{ blog };
```

Использование экспортированных переменных в MDX

MDX поддерживает использование **export** операторов для добавления переменных в содержимое **MDX** или для экспорта данных в компонент, который их импортирует.

Например, вы можете экспортировать **title** поле со страницы или компонента **MDX**, чтобы использовать его в качестве заголовка с помощью **{JSX expressions}**:

- `/src/blog/posts/post-1.mdx`

```
export const title = 'My  
first MDX post'  
  
# {title}
```

Или вы можете использовать экспортированные данные **title** на своей странице, используя операторы **import** **and** **import.meta.glob()**:

- `src/pages/index.astro`

```
---  
const matches =  
import.meta.glob('./posts/*  
.mdx', { eager: true });  
const posts =  
Object.values(matches);  
---  
  
{posts.map(post => <p>  
{post.title}</p>)}
```

Экспортируемые объекты недвижимости

.astro При использовании **import** оператора `<script>` компоненту доступны следующие свойства **import.meta.glob()**:

- **file**- Абсолютный путь к файлу (например,

/home/user/projects/.../file.mdx).

- **url**- URL страницы (например, /en/guides/markdown-content).
- **frontmatter**- Содержит любые данные, указанные в метаданных файла в формате YAML/TOML.
- **getHeadings()**- Асинхронная функция, возвращающая массив всех заголовков (<h1> от до

) в файле с типом: { depth: number; slug: string; text: string }[]. Идентификатор каждого заголовка slug соответствует сгенерированному ID для данного заголовка и может использоваться в качестве якорных ссылок. <Content /> - Компонент, возвращающий полное, отрендеренное содержимое файла. (любое export значение) -
Файлы MDX также могут экспортировать данные с помощью export оператора.

Использование переменных метаданных в MDX

Интеграция **Astro MDX** по умолчанию поддерживает использование метаданных (**frontmatter**) в **MDX**.

Добавляйте свойства метаданных так же, как и в файлах **Markdown**, и эти переменные будут доступны для использования в шаблоне, а также в качестве именованных свойств при импорте файла в другое место.

- `/src/blog/posts/post-1.mdx`

```
---  
title: 'My first MDX post'  
author: 'Houston'  
---  
  
# {frontmatter.title}
```

Written by:
{frontmatter.author}

Использование компонентов в MDX

После установки интеграции **MDX** вы можете импортировать и использовать как компоненты **Astro** , так и компоненты фреймворка пользовательского интерфейса в файлах **MDX (.mdx)** так же, как и в любых других компонентах **Astro**.

Не забудьте добавить его ***client:directive*** в компоненты вашего **UI**-фреймворка, если это необходимо!

Дополнительные примеры использования операторов импорта и экспорта см. в документации **MDX** .

- src/blog/post-1.mdx

```
---  
title: My first post  
---  
import ReactCounter from  
'../components/ReactCounter  
.jsx';
```

I just started my new Astro
blog!

Here is my counter
component, working in MDX:
<ReactCounter client:load
>

**Назначение пользовательских
компонентов элементам HTML**

С помощью **MDX** вы можете сопоставлять синтаксис **Markdown** с пользовательскими компонентами вместо их стандартных **HTML**-элементов. Это позволяет писать код в стандартном синтаксисе **Markdown**, но применять специальные стили к выбранным элементам.

Например, вы можете создать **Blockquote.astro** компонент для настройки стиля **<blockquote>** контента:

- `src/components/Blockquote.astro`

```
---
const props = Astro.props;
---
<blockquote {...props}
class="bg-blue-50 p-4">
  <span class="text-4xl
text-blue-600 mb-
2">"</span>
```

```
<slot /> <!-- Be sure to  
add a `<slot/>` for child  
content! -->  
</blockquote>
```

Импортируйте свой пользовательский компонент в **.mdx** файл, затем экспортируйте **components** объект, который сопоставляет стандартный **HTML**-элемент с вашим пользовательским компонентом:

- `src/blog/posts/post-1.mdx`

```
import Blockquote from  
'../components/Blockquote.a  
stro';  
export const components =  
{blockquote: Blockquote}
```

> This quote will be a custom Blockquote

Полный список **HTML**-элементов, которые можно переопределить в качестве пользовательских компонентов, можно найти на веб-сайте **MDX** .

- *** Примечание

Пользовательские компоненты, определенные и экспортированные в **MDX**-файле, всегда должны быть импортированы, а затем переданы обратно компоненту **<Content />** через **components** соответствующее свойство.

Передача components содержимого в формат MDX

При отображении импортированного **MDX**-контента с помощью **<Content />** компонента, включая отображение **MDX**-

записей с использованием коллекций контента, пользовательские компоненты могут передаваться через свойство **components**. Эти компоненты необходимо предварительно импортировать, чтобы сделать их доступными для **<Content />** компонента.

Этот **components** объект сопоставляет имена **HTML**-элементов (**h1, h2, blockquote, и т. д.**) с вашими пользовательскими компонентами. Вы также можете включить все компоненты, экспортированные из самого **MDX**-файла, используя оператор расширения (...), который также необходимо импортировать из вашего **MDX**-файла как **components**.

Если вы импортируете **MDX**-файл напрямую из одного файла для

использования в компоненте **Astro**,
импортируйте как сам **Content** компонент,
так и любые экспортированные
компоненты из вашего **MDX**-файла.

- `src/pages/page.astro`

```
---  
import { Content,  
components } from  
'../content.mdx';  
import Heading from  
'../Heading.astro';  
---  
<!-- Creates a custom <h1>  
for the # syntax, _and_  
applies any custom  
components defined in  
`content.mdx` -->  
<Content components=  
{...components, h1:  
Heading }> />
```


Если ваш **MDX**-файл является записью из коллекции контента, используйте функцию **render()** для **astro:content** доступа к **<Content />** компоненту.

В следующем примере компоненту передается пользовательский заголовок **<Content />** через **components** свойство, который будет использоваться вместо всех **<h1> HTML**-элементов:

- `src/pages/blog/post-1.astro`

```
---
import { getEntry, render }
from 'astro:content';
import CustomHeading from
'../../components/CustomHeading.astro';
const entry = await
getEntry('blog', 'post-1');
```

```
const { Content } = await
render(entry);
---
<Content components={{ h1:
CustomHeading }} />
```

Конфигурация

После установки интеграции **MDX** для использования **.mdx** файлов в вашем проекте **Astro** не требуется никакой дополнительной настройки.

Вы можете настроить способ отображения **MDX**-файла с помощью следующих параметров:

Параметры, унаследованные от конфигурации **Markdown**.

extendMarkdownConfig **recmaPlugins**
optimize

Параметры, унаследованные от конфигурации **Markdown**.

Все **markdown** параметры конфигурации можно настроить отдельно в интеграции **MDX**. Это включает в себя плагины **remark** и **rehype**, подсветку синтаксиса и многое другое. По умолчанию будут использоваться параметры из вашей конфигурации **Markdown** (см. **extendMarkdownConfig** опцию для изменения этих параметров).

- astro.config.mjs

```
import { defineConfig }  
from 'astro/config';  
import mdx from  
'@astrojs/mdx';  
import remarkToc from  
'remark-toc';
```

```
import rehypePresetMinify
from 'rehype-preset-
minify';

export default
defineConfig({
  // ...
  integrations: [
    mdx({
      syntaxHighlight:
'shiki',
      shikiConfig: { theme:
'dracula' },
      remarkPlugins:
[remarkToc],
      rehypePlugins:
[rehypePresetMinify],
      remarkRehype: {
footnoteLabel: 'Footnotes'
},
      gfm: false,
    }),
  ],
});
```

- ***Осторожно

MDX не поддерживает передачу плагинов **remark** и **rehype** в виде строки. Вместо этого следует установить, импортировать и применить функцию плагина.

Полный список параметров см. в справочнике по параметрам **Markdown** .

extendMarkdownConfig

Тип: boolean По умолчанию: true

Добавлено в: @astrojs/mdx@0.15.0 **MDX** по умолчанию расширяет существующую конфигурацию **Markdown** вашего проекта . Чтобы переопределить отдельные параметры, вы можете указать их эквиваленты в конфигурации **MDX**.

Например, предположим, вам нужно отключить **Markdown**, адаптированный под **GitHub**, и применить другой набор плагинов **remark** для **MDX**-файлов. Вы можете применить эти параметры следующим образом, при этом **extendMarkdownConfig** по умолчанию они включены:

- astro.config.mjs

```
import { defineConfig }
from 'astro/config';
import mdx from
 '@astrojs/mdx';

export default
defineConfig({
  // ...
  markdown: {
    syntaxHighlight:
      'prism',
```

```
    remarkPlugins:
[remarkPlugin1],
    gfm: true,
  },
  integrations: [
    mdx({
      // `syntaxHighlight`
inherited from Markdown

      // Markdown
`remarkPlugins` ignored,
      // only
`remarkPlugin2` applied.
      remarkPlugins:
[remarkPlugin2],
      // `gfm` overridden
to `false`
      gfm: false,
    }),
  ],
});
```

Возможно, вам также потребуется отключить **markdown** расширение конфигурации в **MDX**. Для этого установите **extendMarkdownConfig** значение **false**:

- astro.config.mjs

```
import { defineConfig }
from 'astro/config';
import mdx from
 '@astrojs/mdx';

export default
defineConfig({
  // ...
  markdown: {
    remarkPlugins:
[remarkPlugin1],
  },
  integrations: [
    mdx({
```



```
        // Markdown config
now ignored
        extendMarkdownConfig:
false,
        // No `remarkPlugins`
applied
        }),
    ],
  });
```

remarkPlugins

Тип: PluggableList По умолчанию: []

Добавлено в: @astrojs/mdx@0.11.5

Это плагины, которые напрямую изменяют выходной поток estree . Это полезно для изменения или внедрения переменных **JavaScript** в ваши **MDX**-файлы.

Мы предлагаем использовать **AST Explorer** для экспериментов с результатами поиска в **estree** и попробовать **estree-util-visit** выполнить поиск по узлам **JavaScript**.

optimize

Тип: `boolean | { ignoreElementNames?: string[] }` По умолчанию: `false`

Добавлено в: `@astrojs/mdx@0.19.5`

Это необязательный параметр конфигурации для оптимизации вывода **MDX**-кода с целью ускорения сборки и рендеринга с помощью встроенного плагина **rehype**. Это может быть полезно, если у вас много **MDX**-файлов и вы замечаете медленную сборку. Однако этот параметр может генерировать неэкранированный **HTML**-код, поэтому

убедитесь, что интерактивные части вашего сайта по-прежнему корректно работают после его включения.

Эта функция отключена по умолчанию. Чтобы включить оптимизацию **MDX**, добавьте следующее в конфигурацию интеграции **MDX**:

- astro.config.mjs

```
import { defineConfig }
from 'astro/config';
import mdx from
 '@astrojs/mdx';

export default
defineConfig({
  // ...
  integrations: [
    mdx({
      optimize: true,
```

```
    }),  
  ],  
});
```

ignoreElementNames

Тип: `string[]`

Добавлено в: `@astrojs/mdx@3.0.0`

Ранее известный как

customComponentNames.

Необязательное свойство, **optimize** позволяющее предотвратить обработку оптимизатором **MDX** определенных имен элементов, таких как пользовательские компоненты, передаваемые в импортируемый контент **MDX** через свойство **components** .

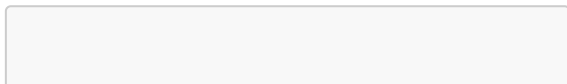
Вам потребуется исключить эти компоненты из оптимизации, поскольку оптимизатор немедленно преобразует контент в статическую строку, что нарушит работу пользовательских компонентов, которые должны динамически отображаться.

Например, предполагаемый вывод **MDX** в следующем коде будет выглядеть:

```
<Heading>...</Heading>
```

так: вместо каждого

```
<h1>...</h1>
```



```
---
import { Content,
components } from
'../content.mdx';
import Heading from
'../Heading.astro';
---

<Content components={{
...components, h1: Heading
}} />
```

Для настройки оптимизации с помощью этого **ignoreElementNames** свойства укажите массив имен **HTML**-элементов, которые следует рассматривать как пользовательские компоненты:

- astro.config.mjs

```
import { defineConfig }
from 'astro/config';
import mdx from
 '@astrojs/mdx';

export default
defineConfig({
  // ...
  integrations: [
    mdx({
      optimize: {
        // Prevent the
optimizer from handling
`h1` elements
        ignoreElementNames:
['h1'],
      },
    }),
  ],
});
```

Обратите внимание, что если ваш **MDX**-файл настраивает пользовательские компоненты с помощью параметра `--config`export const components = { ... }`, то вам не нужно вручную настраивать этот параметр. Оптимизатор обнаружит их автоматически.

Примеры В стартовом шаблоне **Astro** MDX показано, как использовать файлы MDX в вашем проекте Astro.

requestIdleCallback

`requestIdleCallback` — это функция, которая позволяет браузеру выполнять не критичные задачи в моменты, когда у него появляется "свободное время"

по сути это задачи в `EventLoop` но с низким приоритетом и если появится новая задача во время выполнения

старой важной задачи он отложит эту низкую задачу и перейдет к ней только когда освободится от всех важных задач.

Пример:

Порядок_получения	
Порядок_выполнения	Задача
1	1
Важная_№_1	
2	3
НЕ_Важная_№_01	
3	2
Важная_№_2	
4	4
НЕ_Важная_№_02	

- Если вдруг прилетит важная задача во время выполнения неважной он бросит неважную и займется важной... просто все заморозит а

продолжит как выполнит важную...
по этому если вдруг между задачей
важная 1 и важная 2 был
промежуток он занимался
неважной 01 (возможно не до конца
и заморозил доделывание на
свободное время)

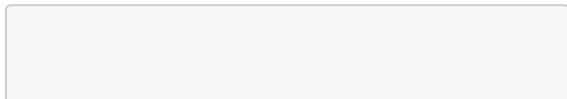
requestIdleCallback-method

Этот `window.requestIdleCallback()` метод ставит в очередь функцию для вызова в периоды простоя браузера. Это позволяет разработчикам выполнять фоновые и низкоприоритетные задачи в основном потоке, не влияя на критически важные с точки зрения задержки события, такие как анимация и ответ на ввод. Функции, как правило, вызываются в порядке поступления (first-in-first-out); однако, функции обратного вызова, для которых

указан таймаут, `timeout` могут вызываться вне порядка, если это необходимо, чтобы выполнить их до истечения таймаута.

Внутри функции обратного вызова в режиме ожидания можно вызвать функцию `requestIdleCallback()`, которая запланирует следующий обратный вызов, который должен произойти не раньше следующего прохода цикла событий.

- **Примечание:** Настоятельно рекомендуется использовать соответствующую `timeout` опцию для выполнения необходимых задач, так как в противном случае может пройти несколько секунд, прежде чем будет вызвана функция обратного вызова.



```
requestIdleCallback(callback)  
requestIdleCallback(callback, options)
```

callback

Ссылка на функцию, которая должна быть вызвана в ближайшем будущем, когда цикл событий находится в режиме ожидания. В функцию обратного вызова передается объект, **IdleDeadline** описывающий доступное время и то, была ли функция обратного вызова выполнена, поскольку истек период ожидания.

options Необязательный

Содержит необязательные параметры конфигурации. В настоящее время

определено только одно свойство:

timeout Если прошло количество миллисекунд, указанное этим параметром, и функция обратного вызова еще не была вызвана, то задача по выполнению этой функции обратного вызова ставится в очередь в цикле событий (даже если это может негативно повлиять на производительность).

timeout Значение параметра должно быть положительным, иначе оно игнорируется.

Возвращаемое значение

Идентификатор, который можно использовать для отмены обратного вызова, передав его в `window.cancelIdleCallback()` метод.

`addclientdirective-option`

addClientDirective — это функция в **API** интеграций **Astro**, которая позволяет разработчикам добавлять собственные директивы клиента (**client directives**) в свой проект.

Простыми словами:

- Директивы клиента (**client:**) в **Astro** (например, **client:load**, **client:idle**) определяют, когда и как интерактивный (клиентский) **JavaScript**-код должен загружаться и выполняться на странице.
- Функция **addClientDirective** дает вам возможность создавать свои собственные, уникальные правила для этого процесса.

Вместо того чтобы полагаться только на стандартные директивы **Astro**, вы можете

определить свою собственную логику, например, "загрузить этот компонент, когда пользователь прокрутит страницу до определенного места" или "загрузить этот код только при определенном условии, которое я сам укажу".

Зачем это нужно:

- Гибкость: Позволяет точно контролировать стратегию загрузки **JavaScript** для конкретных нужд вашего проекта, когда стандартных директив недостаточно.
- Производительность: Помогает отложить загрузку не критичного **JavaScript** до тех пор, пока он действительно не понадобится пользователю, что ускоряет первоначальную загрузку страницы.

- **Расширяемость:** Это инструмент для разработчиков интеграций **Astro**, позволяющий расширять базовую функциональность фреймворка

Тип: (directive: ClientDirectiveConfig) => void;

Добавляет пользовательскую директиву клиента для использования в **.astro** файлах.

Обратите внимание, что точки входа директив объединяются только через **esbuild** и должны быть небольшими, чтобы не замедлять процесс гидратации компонентов.

Пример использования:

- astro.config.mjs


```
import { defineConfig }
from 'astro/config';
import clickDirective from
'./astro-click-
directive/register.js'

//
https://astro.build/config
export default
defineConfig({
  integrations: [
    clickDirective() // Мы
    активируем нашу самодельную
    интеграцию/директиву
  ],
});
```

- astro-click-directive/register.js

```
/**
 * @type {() =>
```

```
import('astro').AstroIntegration}
*/
export default () => ({
  name: "client:click", //
  // Имя нашей директивы в коде:
  client:click
  hooks: {
    "astro:config:setup":
    ({ addClientDirective }) =>
    {
      // Вызываем
      // специальную функцию Astro
      addClientDirective({
        name: "click", //
        // Ключевое слово, которое
        // будет использоваться после
        `client:` (client:click)
        entrypoint:
        `./astro-click-
        directive/click.js`, //
        // Путь к файлу с логикой
        // активации
      });
    }
  }
});
```

```
    },  
  },  
});
```

- Здесь мы говорим Astro: "Когда увидишь атрибут `client:click` на каком-либо компоненте, используй логику из файла **`./astro-click-directive/click.js`**."
- `astro-click-directive/click.js`

```
/**  
 * Hydrate on first click  
on the window  
 * @type  
{import('astro').ClientDire  
ctive}  
 */  
export default (load, opts,  
el) => {
```

```
// Добавляем слушатель  
события на весь документ/  
окно
```

```
window.addEventListener('click', async () => {  
    // *** КОД, КОТОРЫЙ  
    ВЫЗЫВАЕТСЯ ПРИ КЛИКЕ ***
```

```
    // [1] load()  
    // Эта функция  
    возвращает промис, который  
    резолвится в функцию  
    гидратации.
```

```
    const hydrate = await  
    load()
```

```
    // [2] hydrate()  
    // Эта функция  
    фактически запускает  
    JavaScript вашего  
    компонента (например,  
    монтирует React-  
    приложение).
```

```
    await hydrate()  
  
    }, { once: true }) // {  
    once: true } гарантирует,  
    что слушатель сработает  
    только один раз, а затем  
    удалится.  
  }
```

- Подробное объяснение `export default (load, opts, el) => { ... }`:

Когда **Astro** видит компонент с ***client:click***, оно вызывает эту функцию, передавая ей три аргумента:

- **load** (функция): Это обещание загрузить и скомпилировать **JavaScript** вашего компонента из сети. Это асинхронная операция.

Когда вы вызываете `const hydrate = await load()`, вы говорите: "Загрузи **JS**-файл компонента сейчас".

- **opts** (объект): Дополнительные опции, которые вы могли бы передать (в этом примере не используются).
- **el** (**DOM**-элемент): Ссылка на **HTML**-элемент, на котором висит атрибут ***client:click***.

Что делает этот блок кода: Он ждет первого клика на странице. Только в этот момент он запускает процесс `load()`, чтобы **загрузить JS** компонента, а затем запускает `hydrate()`, чтобы компонент стал интерактивным.

Переделанный пример с выводом в консоль

Изменим файл **astro-click-directive/click.js**, чтобы он писал сообщения в консоль, делая процесс более наглядным.

- astro-click-directive/click.js
(Измененный вариант)

```
/**
 * Hydrate on first click
on the window
 * @type
{import('astro').ClientDire
ctive}
 */
export default (load, opts,
el) => {
  console.log('>>> Astro: Я
жду первого клика на
странице, чтобы загрузить
компонент...');
}
```

```
window.addEventListener('click', async () => {  
    console.log('>>> КЛИК  
ЗАФИКСИРОВАН! Начинаю  
загрузку JavaScript...');  
  
    // 1. Загружаем код  
    const hydrate = await  
load();  
    console.log('>>>  
JavaScript загружен. Сейчас  
буду его выполнять  
(гидратация)...');  
  
    // 2. Выполняем код  
    await hydrate();  
    console.log('>>>  
Компонент теперь  
интерактивен!');  
  
}, { once: true });  
};
```


Как это использовать в компоненте

Можете использовать это так в любом файле **.astro** с вашим интерактивным фреймворком (например, React, Vue):

- MyInteractiveComponent.jsx (React)

```
import React, { useState }
from 'react';

export default function
MyInteractiveComponent() {
  const [count, setCount] =
useState(0);

  // Этот код не будет
  выполняться до тех пор,
  пока кто-то не кликнет на
  странице
  // и не сработает наша
  директива client:click.
```

```
return (  
  <div>  
    <p>Счетчик: {count}  
  </p>  
    <button onClick={()  
=> setCount(count + 1)}>  
      Кликни меня  
    </button>  
  </div>  
);  
}
```

- В файле .astro

```
<MyInteractiveComponent  
  client:click />
```

ТИПЫ

Вы также можете добавить типы для директив в файл определения типов вашей библиотеки:

- `astro-click-directive/index.d.ts`

```
import 'astro'
declare module 'astro' {
  interface
  AstroClientDirectives {
    'client:click'?:
    boolean
  }
}
```

server-islands

«Серверные острова» (Server Islands) в Astro — это архитектурный подход, позволяющий создавать очень быстрые по умолчанию веб-сайты, добавляя

небольшие, динамически обновляемые «островки» контента, которые рендерятся на сервере, только там, где это действительно нужно.

Зачем это нужно (простыми словами)

Основная идея состоит в том, чтобы обеспечить наилучшую производительность и пользовательский опыт (**User Experience**).

- Скорость загрузки: Большая часть вашей страницы — это статический **HTML**, который загружается мгновенно, потому что он может быть закэширован на **CDN** (сеть доставки контента). Это обеспечивает быструю первую отрисовку страницы (**First Contentful Paint**).

- Гибридный подход: Вы получаете лучшее от статических сайтов (скорость и надежность) и динамических сайтов (актуальность данных).
- Меньше **JavaScript**: На клиентскую сторону (в браузер пользователя) отправляется минимум **JavaScript**-кода, что снижает нагрузку на устройство пользователя и ускоряет работу сайта.

Как это работает:

Представьте, что у вас есть блог. Большая часть страницы (статьи, заголовки, меню) статична и не меняется. Но некоторые элементы требуют актуальной информации:

- Аватар пользователя (после входа в систему).
- Количество товаров в корзине.
- Рекомендации на основе ваших предыдущих просмотров.
- Цены в реальном времени.

Вместо того чтобы делать всю страницу динамической и заставлять сервер обрабатывать ее целиком при каждом запросе, вы помечаете только эти конкретные элементы как «серверные острова» с помощью директивы ***server:defer.***

- Браузер быстро загружает основную статическую страницу.
- В местах, где должны быть «островки», он видит заглушку (**fallback**).

- Отдельным запросом браузер получает актуальный **HTML**-код для этих «островков» с сервера, и они встраиваются на страницу уже после ее загрузки, не замедляя основной контент.

Серверные острова

Серверные острова позволяют по запросу отображать динамические или персонализированные «острова» по отдельности, не жертвуя при этом производительностью остальной части страницы.

Это означает, что посетитель увидит наиболее важные части вашей страницы быстрее, а также позволит более эффективно кэшировать основной

контент, обеспечивая более высокую скорость работы.

Компоненты серверного острова

Серверный остров — это обычный компонент **Astro**, отрисовываемый на сервере, которому дается указание отложить отрисовку до тех пор, пока его содержимое не станет доступно.

Ваша страница будет немедленно отрисована с любым указанным резервным содержимым в качестве заполнителя. Затем собственное содержимое компонента будет загружено на стороне клиента и отображено, когда оно станет доступно.

После установки адаптера для отложенной отрисовки добавьте ***server:defer*** директиву к любому

компоненту на вашей странице, чтобы превратить его в самостоятельный «островок»:

- `src/pages/index.astro`

```
---  
import Avatar from  
'../components/Avatar.astro'  
;  
---  
  
<Avatar server:defer />
```

Эти компоненты могут выполнять любые действия, которые обычно выполняются на странице, отображаемой по запросу с помощью адаптера, например, получать контент и получать доступ к файлам **cookie**:

- `src/components/Avatar.astro`

```
---  
import { getUserAvatar }  
from '../sessions';  
const userSession =  
Astro.cookies.get('session'  
);  
const avatarURL = await  
getUserAvatar(userSession);  
---  
<img alt="User avatar" src=  
{avatarURL} />
```

Передача ресурсов на серверные острова.

Свойства, передаваемые компонентам серверного острова, должны быть **сериализуемыми**: способными быть преобразованы в формат, подходящий для передачи по сети или в хранилище.

Кроме того, **Astro** не сериализует все типы сериализуемых структур данных. Поэтому существуют некоторые ограничения на то, что можно передавать в качестве свойств серверному острову.

Следует отметить, что функции нельзя передавать компонентам, помеченным **server:defer** как , поскольку они не подлежат сериализации. Объекты с циклическими ссылками также не сериализуются.

Поддерживаются следующие типы свойств : простой объект number, и string
Array Map Set Reg Exp Date Big Int URL
Uint8Array Uint16Array Uint32Array Infinity

Резервный контент для серверного острова

При использовании **server:defer** атрибута компонента для задержки его рендеринга вы можете «вставить» содержимое загрузки по умолчанию, используя включенный именованный **"fallback"** слот.

Резервный контент будет отображаться вместе с остальной частью страницы при первоначальной загрузке страницы и будет заменен содержимым вашего компонента, когда оно станет доступно.

Для добавления резервного контента добавьте **slot="fallback"** дочерний элемент (другой компонент или **HTML**-элемент), переданный компоненту **server island**:

```
---  
import Avatar from
```

```
'../components/Avatar.astro';
import GenericAvatar from
'../components/GenericAvatar.astro';
---
<Avatar server:defer>
  <GenericAvatar
slot="fallback" />
</Avatar>
```

В качестве резервного контента могут выступать, например, следующие элементы:

- Вместо пользовательского аватара используется стандартный.
- Заполнитель пользовательского интерфейса, например, для пользовательских сообщений.
- Индикаторы загрузки, такие как вращающиеся элементы.

Как это работает

Реализация серверной островной архитектуры происходит в основном на этапе сборки, когда содержимое компонентов заменяется небольшим скриптом.

Каждый из островов, отмеченных этим символом, **server:defer** выделен в отдельный маршрут, который скрипт загружает во время выполнения. При сборке вашего сайта **Astro** пропустит этот компонент и внедрит вместо него скрипт, а также любой контент, отмеченный вами этим символом ***slot="fallback"***.

При загрузке страницы в браузере эти компоненты будут отправлены на специальную конечную точку, которая их отобразит и вернет **HTML**-код. Это

означает, что пользователи мгновенно увидят наиболее важные части страницы. Резервный контент будет виден в течение короткого времени, после чего загрузятся динамические элементы.

Каждый остров загружается независимо от остальных. Это значит, что более медленная загрузка острова не повлияет на доступность остального персонализированного контента.

Этот шаблон рендеринга разработан с учетом возможности переноса. Он не зависит от какой-либо серверной инфраструктуры, поэтому будет работать с любым вашим хостом, от сервера **Node.js** в контейнере **Docker** до любого выбранного вами бессерверного провайдера.

Кэширование

Данные для серверных островов извлекаются посредством **GET** запроса, при этом свойства передаются в виде зашифрованной строки в **URL**-адресе. Это позволяет кэшировать данные с помощью **Cache-Control** HTTP-заголовка, используя стандартные **Cache-Control** директивы.

Однако браузер ограничивает **максимальную длину URL-адресов 2048 байтами** по практическим соображениям и во избежание проблем, приводящих к отказу в обслуживании. Если строка запроса приводит к превышению этого лимита, **Astro** вместо этого отправит **POST** запрос, содержащий все свойства в теле запроса.

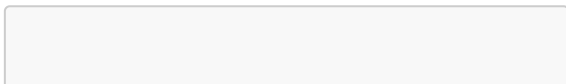
POST Запросы не кэшируются браузерами, поскольку используются для отправки данных и могут вызывать проблемы с целостностью данных или безопасностью. Поэтому любая существующая логика кэширования в вашем проекте перестанет работать. По возможности передавайте на сервер только необходимые свойства и избегайте отправки целых объектов данных и массивов, чтобы ваш запрос оставался небольшим.

Доступ к URL-адресу страницы в изолированном серверном модуле.

В большинстве случаев вы, как и в обычных компонентах, можете получить информацию о отображаемой странице, передавая свойства (props) .

Однако серверные островки работают в собственном изолированном контексте вне запроса страницы. **Astro.url** И **Astro.request.url** в компоненте серверного островка оба возвращают **URL**-адрес, который выглядит как **/_server-islands/Avatar** вместо **URL**-адреса текущей страницы в браузере. Кроме того, если вы предварительно отрисовываете страницу, у вас не будет доступа к такой информации, как параметры запроса, для передачи в качестве свойств.

Чтобы получить доступ к информации из **URL**-адреса страницы, вы можете проверить заголовок **Referer** , который будет содержать адрес страницы, загружающей остров в браузере:



```
---  
const referer =  
Astro.request.headers.get('Referer');  
const url = new  
URL(referer);  
const productId =  
url.searchParams.get('product');  
---
```

Повторное использование ключа шифрования

Astro использует *криптографию* для шифрования свойств, передаваемых на серверные острова, защищая конфиденциальные данные от случайного разглашения. Это шифрование основано на новом случайном ключе, который

генерируется при каждой сборке и внедряется в серверный пакет.

Большинство хостингов для развертывания автоматически обеспечивают синхронизацию фронтенда и бэкенда. Однако вам может потребоваться постоянный ключ шифрования, если вы используете поэтапное развертывание, многорегиональный хостинг или **CDN**, кэширующий страницы, содержащие изолированные серверы.

В средах с поэтапным развертыванием (например, **Kubernetes**), где ваши фронтенд-ресурсы (которые шифруют свойства) и ваши бэкенд-функции (которые расшифровывают свойства) могут временно использовать разные ключи, или когда **CDN** все еще

обслуживает страницы, созданные с использованием старого ключа, зашифрованные свойства, передаваемые на ваш серверный остров, не могут быть расшифрованы.

В подобных ситуациях используйте **Astro CLI** для генерации многоразового закодированного ключа шифрования, который можно установить в качестве переменной среды в вашей среде сборки:

```
astro create-key
```

Используйте это значение для настройки **ASTRO_KEY** переменной среды (например, в **.env** файле) и включите его в параметры сборки вашей системы **CI/CD** или хоста. Это гарантирует, что один и тот же ключ всегда будет использоваться в

сгенерированном пакете, так что шифрование и дешифрование будут оставаться синхронизированными.

Стили и CSS

Astro был разработан, чтобы сделать стилизацию и написание **CSS** простым делом. Пишите свой собственный **CSS** напрямую внутри **Astro**-компонента или импортируйте вашу любимую **CSS**-библиотеку, например **Tailwind**.

Продвинутые языки стилизации, такие как **Sass** и **Less**, также поддерживаются.

Стилизация в Astro

Стилизация компонента в **Astro** так же проста, как добавление тега **<style>** в ваш шаблон компонента или страницы. Когда вы размещаете тег **<style>** внутри Astro-

компонента, Astro автоматически обнаружит CSS и обработает ваши стили.

- `src/components/MyComponent.astro`

```
<style>
  h1 { color: red; }
</style>
```

Локальные стили

Правила **CSS** в теге **<style> Astro** по умолчанию автоматически обладают ограниченной областью видимости. Подобные стили компилируются таким образом, что применяются только к **HTML**-коду внутри этого же компонента. **CSS**, написанный внутри **Astro**-компонента, автоматически

инкапсулируется в рамках этого компонента.

Вот этот CSS:

- src/pages/index.astro

```
<style>
  h1 {
    color: red;
  }

  .text {
    color: blue;
  }
</style>
```

Компилируется в этот:

```
<style>
  h1[data-astro-cid-
```



```
hnhqfkh6] {  
    color: red;  
}  
  
.text[data-astro-cid-  
hnhqfkh6] {  
    color: blue;  
}  
</style>
```

Локальные стили не распространяются за пределы компонента и не влияют на остальные части вашего сайта. В **Astro** можно безопасно использовать селекторы с низкой специфичностью, такие как **h1 {}** или **p {}**, поскольку в финальной сборке они будут автоматически преобразованы с учётом ограниченной области видимости.

Локальные стили также не применяются к другим **Astro**-компонентам внутри вашего шаблона. Если вам нужно стилизовать дочерний компонент, рекомендуется обернуть его в `<div>` (или другой элемент), который затем можно стилизовать.

Специфичность локальных стилей сохраняется, что позволяет им корректно работать вместе с другими **CSS**-файлами или **CSS**-библиотеками, при этом сохраняя чёткие границы, предотвращающие применение стилей за пределами компонента.

global-styles

Глобальные стили

Хотя мы рекомендуем использовать локальные стили для большинства компонентов, иногда могут возникнуть ситуации, когда требуется написать глобальный **CSS**. Можно отключить автоматическую ограниченную область видимости с помощью атрибута **<style is:global>**.

- `src/components/GlobalStyles.astro`

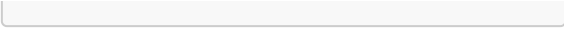
```
<style is:global>
  /* Без ограниченной
  области видимости, стили
  доставляются в браузер как
  есть.

  Применяется ко всем
  тегам `<h1>` на вашем
  сайте. */
  h1 { color: red; }
</style>
```

Вы также можете сочетать глобальные и локальные правила **CSS** в одном теге **<style>**, используя селектор **:global()**. Это становится мощным инструментом для применения стилей к дочерним элементам вашего компонента.

- src/components/MixedStyles.astro

```
<style>
  /* Стили только для этого
  компонента. */
  h1 { color: red; }
  /* Смешанный режим:
  применяется только к
  дочерним элементам `h1`. */
  article :global(h1) {
    color: blue;
  }
</style>
<h1>Заголовок</h1>
<article><slot /></article>
```



Это отличный способ стилизации таких элементов, как посты блога или документы с контентом из **CMS**, где содержимое находится вне **Astro**. Однако будьте осторожны: компоненты, внешний вид которых меняется в зависимости от наличия определённого родительского компонента, могут вызвать сложности при отладке.

Локальные стили следует использовать как можно чаще. Глобальные стили стоит применять только при необходимости.

Комбинирование классов с `class:list`

Если вам нужно динамически комбинировать классы на элементе, используйте служебный атрибут **class:list** в **.astro** файлах.

- src/components/ClassList.astro

```
---
const { isRed } =
  Astro.props;
---
<!-- Если `isRed` истинно,
класс будет "box red". -->
<!-- Если `isRed` ложно,
класс будет "box". -->
<div class:list={['box', {
  red: isRed }]}><slot />
</div>

<style>
  .box { border: 1px solid
blue; }
  .red { border-color: red;
}
</style>
```

[Подробнее о class:list.](#)

CSS-переменные

Добавлено в: astro@0.21.0

Тег **<style>** в **Astro** может использовать любые **CSS**-переменные, доступные на странице. Вы также можете передавать **CSS**-переменные напрямую из метаданных компонента с помощью директивы **define:vars**.

- src/components/DefineVars.astro

```
---
const foregroundColor =
  "rgb(221 243 228)";
const backgroundColor =
  "rgb(24 121 78)";
---
<style define:vars={{
  foregroundColor,
  backgroundColor }}>
```

```
h1 {  
  background-color: var(--  
background-color);  
  color: var(--  
foreground-color);  
}  
</style>  
<h1>Hello</h1>
```

[Подробнее о define:vars.](#)

Передача class дочернему компоненту

В **Astro HTML**-атрибуты, такие как **class**, не передаются дочерним компонентам автоматически.

Вместо этого необходимо:

- Принять пропс class в дочернем компоненте

- Применить его к корневому элементу
- При деструктуризации переименовать его, так как `class` — **зарезервированное слово** в **JavaScript**.

При использовании стратегии локальных стилей по умолчанию необходимо также передавать атрибут **`data-astro-cid-*`**. Это можно сделать, передав остальные пропсы (**`...rest`**) в компонент. Если вы изменили **`scopedStyleStrategy`** на **`'class'`** или **`'where'`**, передача **`...rest`** не требуется.

- `src/components/MyComponent.astro`

```
---  
const { class: className,  
...rest } = Astro.props;  
---
```

```
<div class={className}  
  {...rest}>  
  <slot/>  
</div>
```

- src/pages/index.astro

```
---  
import MyComponent from  
  "../components/MyComponent.  
  astro"  
---  
<style>  
  .red {  
    color: red;  
  }  
</style>  
<MyComponent  
  class="red">ЭТОТ ТЕКСТ  
  будет красным!  
</MyComponent>
```

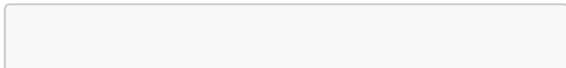
- Локальные стили родительских компонентов

Поскольку атрибут **data-astro-cid-*** включает дочерний компонент в область видимости родителя, стили могут наследоваться от родителя к потомку. Чтобы избежать нежелательных побочных эффектов, используйте уникальные имена классов в дочернем компоненте.

Встроенные стили

Вы можете задавать стили **HTML**-элементам напрямую через атрибут **style**, используя **CSS**-строку или объект со свойствами **CSS**:

- `src/pages/index.astro`



```
// Эти примеры  
эквивалентны:  
<p style={{ color: "brown",  
textDecoration: "underline"  
}}>Мой текст</p>  
<p style="color: brown;  
text-decoration:  
underline;">Мой текст</p>
```

Внешние стили

Подключение глобальных внешних стилей возможно двумя способами:

- Импорт через **ESM** для файлов в исходном коде проекта
- Абсолютная ссылка для файлов в директории **public/** или внешних ресурсов

внешняя ссылка: [Подробнее об использовании статических ресурсов, размещённых в директориях public/ или src/.](#)

Импорт локальной таблицы стилей

- Используете **npm**-пакет? Возможно, вам потребуется обновить **astro.config** при импорте стилей из **npm**-пакетов. См. раздел Импорт стилей из **npm**-пакета ниже.

Вы можете импортировать таблицы стилей в метаданных **Astro**-компонента, используя синтаксис **ESM**-импорта.

Импорт **CSS** работает аналогично другим **ESM**-импортам в **Astro**-компоненте и должен:

- Указываться относительно компонента

- Располагаться в верхней части скрипта компонента
- Быть объявлен вместе с другими импортами
- `src/pages/index.astro`

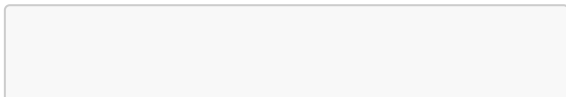
```
---  
// Astro автоматически  
соберет и оптимизирует этот  
CSS за вас  
// Это работает и с файлами  
препроцессоров: .scss,  
.styl и др.  
import  
'../styles/utils.css';  
---  
<html><!-- Здесь ваша  
страница --></html>
```

Импорт **CSS** через **ESM** поддерживается в любом **JavaScript**-файле, включая **JSX**-компоненты (**React**, **Preact** и др.). Это особенно полезно для создания детализированных стилей отдельных компонентов **React**.

Импорт стилей из npm-пакета

Возможно, вам потребуется загрузить стили из внешнего **npm**-пакета. Это особенно актуально для таких утилит, как **Open Props**. Если пакет рекомендует указывать расширение файла (например, **package-name/styles.css** вместо **package-name/styles**), импорт будет работать так же, как с локальными стилями:

- `src/pages/random-page.astro`



```
---  
import 'package-  
name/styles.css';  
---  
<html><!-- Здесь ваша  
страница --></html>
```

Если ваш пакет не рекомендует указывать расширение файла (например, **package-name/styles**), сначала потребуется обновить конфигурацию **Astro**!

Допустим, вы импортируете **CSS**-файл **normalize** из пакета **package-name** (без указания расширения файла). Чтобы гарантировать корректный пререндеринг страницы, добавьте **package-name** в массив **vite.ssr.noExternal**:

- astro.config.mjs


```
import { defineConfig }  
from 'astro/config';  
  
export default  
defineConfig({  
  vite: {  
    ssr: {  
      noExternal:  
        ['package-name'],  
    },  
  },  
})
```

- ***Заметка

Это специфичная настройка Vite, которая не связана (и не требует) с SSR в Astro.

Теперь вы можете свободно импортировать **package-name/normalize**. Astro обработает и оптимизирует этот

файл, как и любую другую локальную таблицу стилей.

- `src/pages/random-page.astro`

```
---
import 'package-
name/normalize';
---
<html><!-- Здесь ваша
страница --></html>
```

Подключение статической таблицы стилей через тег `<link>`

Для загрузки CSS-файлов также можно использовать элемент `<link>`. Укажите в нём:

Абсолютный путь к файлу в директории **`/public`** Или **URL** внешнего ресурса

- Ограничение Относительные пути в атрибуте **href** не поддерживаются.
- `src/pages/index.astro`

```
<head>
  <!-- Локальная таблица
стилей:
/public/styles/global.css -
->
  <link rel="stylesheet"
href="/styles/global.css"
/>
  <!-- Внешняя таблица
стилей -->
  <link rel="stylesheet"
href="https://cdn.jsdelivrivr.
net/npm/prismjs@1.24.1/them
es/prism-tomorrow.css" />
</head>
```

Поскольку этот метод использует директорию **public/**, он пропускает стандартную обработку **CSS**, сборку и оптимизацию, которые предоставляет **Astro**. Если вам нужны эти преобразования, используйте метод импорта стилей, описанный выше.

Порядок наследования

Компоненты **Astro** иногда должны учитывать несколько источников **CSS**. Например, ваш компонент может:

- Импортировать внешнюю таблицу стилей
- Содержать собственный тег **<style>**
- И рендериться внутри макета, который тоже импортирует CSS

Когда конфликтующие **CSS**-правила применяются к одному элементу,

браузеры сначала используют специфичность, а затем порядок появления для определения конечного стиля.

Если одно правило более специфично чем другое, его значение будет иметь приоритет — независимо от порядка расположения:

- `src/components/MyComponent.astro`

```
<style>
  h1 { color: red }
  div > h1 {
    color: purple
  }
</style>
<div>
  <h1>
    Этот заголовок будет
    фиолетовым!
```

```
</h1>  
</div>
```

- пометка внешняя ссылка [Как по мне](#)
дичь в mdn описаны приоритеты и в
принципе так работает css и сама
астро к этому ничего не имеет
общего....

Если два правила имеют одинаковую
специфичность, приоритет определяется
по порядку появления — будет
применено значение последнего правила:

- `src/components/MyComponent.astro`

```
<style>  
  h1 { color: purple }  
  h1 { color: red }  
</style>  
<div>
```

```
<h1>
  Этот заголовок будет
красным!
</h1>
</div>
```

Правила CSS в Astro применяются в следующем порядке:

1. Теги **<link>** в **<head>** (наименьший приоритет)
2. Импортированные стили
3. Локальные стили (наибольший приоритет)

Локальные стили

В зависимости от значения **scopedStyleStrategy**, локальные стили могут увеличивать специфичность класса, но не всегда.

Однако локальные стили всегда имеют самый поздний порядок появления. Поэтому они будут иметь приоритет над другими стилями с такой же специфичностью. Например, если импортированный стиль конфликтует с локальным стилем, будет применено значение локального стиля:

- `src/components/make-it-purple.css`

```
h1 {  
  color: purple;  
}
```

- `src/components/MyComponent.astro`

```
---  
import './make-it-  
purple.css'
```



```
---  
<style>  
  h1 { color: red }  
</style>  
<div>  
  <h1>  
    Этот заголовок будет  
    красным!  
  </h1>  
</div>
```

Локальные стили будут перезаписаны, если импортированный стиль имеет более высокую специфичность. В этом случае приоритет отдаётся стилю с наибольшей специфичностью:

- `src/components/make-it-purple.css`

```
#intro {  
  color: purple;
```

```
}
```

- src/components/MyComponent.astro

```
---  
import "./make-it-  
purple.css"  
---  
<style>  
  h1 { color: red }  
</style>  
<div>  
  <h1 id="intro">  
    Этот заголовок будет  
    фиолетовым!  
  </h1>  
</div>
```

Порядок импорта

При импорте нескольких таблиц стилей в компоненте **Astro CSS**-правила обрабатываются в порядке их импорта. Более высокая специфичность всегда определяет, какие стили отображать, независимо от времени обработки **CSS**. Но если конфликтующие стили имеют одинаковую специфичность, побеждает последний импортированный:

- `src/components/make-it-purple.css`

```
div > h1 {  
  color: purple;  
}
```

- `src/components/make-it-green.css`

```
div > h1 {  
  color: green;  
}
```

- src/components/MyComponent.astro

```
---  
import "./make-it-  
green.css"  
import "./make-it-  
purple.css"  
---  
<style>  
  h1 { color: red }  
</style>  
<div>  
  <h1>  
    Этот заголовок будет  
    фиолетовым!  
  </h1>  
</div>
```

Хотя теги **<style>** имеют ограниченную область видимости и применяются только к компоненту, который их объявляет, импортированный **CSS** может «просачиваться». Импорт компонента применяет любой **CSS**, который он импортирует, даже если компонент никогда не используется:

- `src/components/PurpleComponent.astro`

```
---
import './make-it-
purple.css'
---
<div>
  <h1>Импортирую фиолетовый
  CSS.</h1>
</div>
```

- src/components/MyComponent.astro

```
---
import "../make-it-
green.css"
import PurpleComponent from
"./PurpleComponent.astro";
---
<style>
  h1 { color: red }
</style>
<div>
  <h1>
    Этот заголовок будет
    фиолетовым!
  </h1>
</div>
```

- Совет

В Astro распространён подход импорта глобального **CSS** внутри компонента **Layout**. Убедитесь, что импортируете компонент **Layout** до других импортов, чтобы он имел наименьший приоритет.

Подключение стилей через теги `<link>`

Таблицы стилей, загруженные через теги `<link>`, обрабатываются в порядке их подключения, перед любыми другими стилями в **Astro**-файле. Следовательно, эти стили будут иметь более низкий приоритет, чем импортированные таблицы стилей и локальные стили:

- `src/pages/index.astro`

```
---  
import "../components/make-
```

```
it-purple.css"
```

```
---
```

```
<html lang="ru">
```

```
  <head>
```

```
    <meta charset="utf-8"
```

```
    <link rel="icon"
```

```
  type="image/svg+xml"
```

```
  href="/favicon.svg" />
```

```
    <meta name="viewport"
```

```
  content="width=device-  
width" />
```

```
    <meta name="generator"
```

```
  content={Astro.generator}
```

```
    <title>Astro</title>
```

```
    <link rel="stylesheet"
```

```
  href="/styles/make-it-  
blue.css" />
```

```
  </head>
```

```
  <body>
```

```
    <div>
```

```
      <h1>ЭТОТ ЗАГОЛОВОК
```



```
будет фиолетовым!</h1>  
    </div>  
  </body>  
</html>
```

Tailwind

Astro поддерживает популярные **CSS**-библиотеки, инструменты и фреймворки, включая [Tailwind](#) и другие!

Astro работает с **Tailwind** 3 и 4. Вы можете:

- Добавить Tailwind 4 через Vite-плагин с помощью CLI-команды
- Вручную установить зависимости для поддержки Tailwind 3 через интеграцию Astro

Для апгрейда с Tailwind 3 до 4 потребуется:

- Добавить поддержку Tailwind 4
- Удалить поддержку Tailwind 3

Добавление Tailwind 4

В **Astro** **>=5.2.0** используйте команду **astro add tailwind** для вашего менеджера пакетов, чтобы установить официальный плагин **Tailwind** для **Vite**. Для более ранних версий **Astro** следуйте инструкциям в документации **Tailwind**, чтобы вручную добавить плагин **@tailwindcss/vite**.

```
npx astro add tailwind
```

Затем импортируйте **tailwindcss** в **src/styles/global.css** (или любой другой выбранный вами **CSS**-файл), чтобы сделать классы **Tailwind** доступными для

вашего **Astro**-проекта. Этот файл с импортом будет создан автоматически, если вы использовали команду **astro add tailwind** для установки **Vite**-плаги́на.

- `src/styles/global.css`

```
@import "tailwindcss";
```

Импортируйте этот файл на страницах, где должен применяться **Tailwind**. Обычно это делается в компоненте макета, чтобы стили **Tailwind** были доступны на всех страницах, использующих этот макет:

- `src/layouts/Layout.astro`

```
---  
import
```

```
"/styles/global.css";  
---
```

CSS-препроцессоры

Astro поддерживает **CSS**-препроцессоры, такие как **Sass**, **Stylus** и **Less**, через **Vite**.

Sass и SCSS

```
npm install sass
```

Используйте `<style lang="scss">` или `<style lang="sass">` в файлах `.astro`.

Stylus

```
npm install stylus
```

Используйте `<style lang="styl">` или `<style lang="stylus">` в файлах `.astro**`.

Less

```
npm install less
```

Используйте `<style lang="less">` в файлах `.astro`.

LightningCSS

```
npm install lightningcss
```

Обновите вашу конфигурацию **vite** в `astro.config.mjs`:

- astro.config.mjs

```
import { defineConfig }
from 'astro/config'

export default
defineConfig({
  vite: {
    css: {
      transformer:
"lightningcss",
    },
  },
})
```

В компонентах фреймворков

Вы также можете использовать все перечисленные **CSS**-препроцессоры в компонентах **JS**-фреймворков! Следуйте

рекомендациям соответствующего фреймворка:

- React / Preact: import Styles from './styles.module.scss';
- Vue: <style lang="scss">
- Svelte: <style lang="scss">

PostCSS

Astro включает **PostCSS** как часть **Vite**.

Для настройки **PostCSS** создайте файл **postcss.config.cjs** в корне проекта.

Плагины можно импортировать через **require()** после их установки (например **npm install autoprefixer**).

- postcss.config.cjs

```
module.exports = {  
  plugins: [  

```

```
require('autoprefixer'),  
    require('cssnano'),  
  ],  
};
```

Фреймворки и библиотеки

React / Preact

Файлы **.jsx** поддерживают как глобальный **CSS**, так и **CSS**-модули. Для использования **CSS**-модулей применяйте расширение **.module.css** (или **.module.scss/.module.sass** при работе с **Sass**).

- `src/components/MyReactComponent.jsx`


```
import './global.css'; //  
включаем глобальный CSS  
import Styles from  
 './styles.module.css'; //  
используем CSS-модули
```

Vue

Vue в **Astro** поддерживает те же методы, что и **vue-loader**:

vue-loader - Локальный CSS **vue-loader** - CSS-модули

Svelte

Svelte в **Astro** также работает ожидаемым образом: [Документация по стилям Svelte](#).

Стилизация Markdown

Любые методы стилизации **Astro** доступны для компонента **макета Markdown**, но разные методы будут по-разному влиять на стилизацию вашей страницы.

Вы можете применить глобальные стили к содержимому **Markdown**, добавив импортированные таблицы стилей в макет, который оборачивает содержимое страницы. Также возможно стилизовать **Markdown** с помощью тегов `<style is:global>` в компоненте макета.

Обратите внимание, что любые добавленные стили подчиняются порядку наследования **Astro**, и вам следует внимательно проверить отрендеренную страницу, чтобы убедиться, что стили применяются должным образом.

Вы также можете добавить **CSS**-интеграции, включая **Tailwind**. Если вы используете **Tailwind**, для стилизации **Markdown** вам может пригодиться плагин типографики.

Продакшен

Контроль сборки

При сборке сайта для продакшена **Astro** минифицирует и объединяет ваш **CSS** в чанки. Каждая страница сайта получает собственный чанк, а **CSS**, общий для нескольких страниц, выделяется в отдельные чанки для повторного использования.

Однако, когда несколько страниц используют общие стили, некоторые общие чанки могут быть очень

маленькими. Если отправлять их все отдельно, это приведёт к множеству запросов таблиц стилей и повлияет на производительность сайта. Поэтому по умолчанию **Astro** добавляет в **HTML** только те чанки, размер которых превышает 4 КБ, в виде тегов **<link rel="stylesheet">**, в то время как меньшие чанки встраиваются как **<style type="text/css">**. Такой подход обеспечивает баланс между количеством дополнительных запросов и объёмом **CSS**, который можно кэшировать между страницами.

Вы можете настроить минимальный размер (в байтах) для внешнего подключения таблиц стилей с помощью опции сборки **Vite assetsInlineLimit**. Обратите внимание, что эта опция также

влияет на встраивание скриптов и изображений.

- astro.config.mjs

```
import { defineConfig }
from 'astro/config';

export default
defineConfig({
  vite: {
    build: {
      assetsInlineLimit:
1024,
    }
  };
});
```

Если вы предпочитаете, чтобы все стили проекта оставались внешними, вы можете

настроить опцию сборки
inlineStylesheets.

- astro.config.mjs

```
import { defineConfig }  
from 'astro/config';  
  
export default  
defineConfig({  
  build: {  
    inlineStylesheets:  
    'never'  
  }  
});
```

Также можно установить значение
'always' для этой опции, что приведёт к
встраиванию всех таблиц стилей.

Продвинутые настройки

- Осторожно

Будьте осторожны при обходе встроенной системы сборки **CSS** в **Astro**! Стили не будут автоматически включены в сборку, и вы должны самостоятельно обеспечить корректное включение указанного файла в финальный вывод страницы.

Импорт CSS с raw

Для сложных сценариев **CSS** можно загружать напрямую с диска без обработки и оптимизации **Astro**. Это может быть полезно, когда требуется полный контроль над определённым фрагментом **CSS** и необходимо обойти автоматическую обработку стилей в **Astro**.

- ! Этот подход не рекомендуется большинству пользователей.
- `src/components/RawInlineStyles.astro`

```
---  
// Расширенный пример! Не  
рекомендуется для  
большинства пользователей!  
import rawStylesCSS from  
'../styles/main.css?raw';  
---  
<style is:inline set:html=  
{rawStylesCSS}></style>
```

Импорт CSS с ?url

Для сложных сценариев можно импортировать прямую ссылку на **CSS**-файл из директории **src/** вашего проекта. Это может быть полезно, когда требуется

полный контроль над загрузкой **CSS**-файла на странице. Однако это отключит оптимизацию данного **CSS**-файла вместе с остальными стилями страницы.

- ! Этот подход не рекомендуется большинству пользователей. Вместо этого размещайте **CSS**-файлы в директории **public/** для получения постоянной ссылки.
- ! Осторожно

Импорт меньшего **CSS**-файла с помощью **?url** может вернуть содержимое **CSS**-файла, закодированное в формате **base64**, как **URL**-данные в итоговой сборке. Либо пишите код, поддерживающий закодированные **URL**-данные **(data:text/css;base64,...)**, либо

установите параметр конфигурации **vite.build.assetsInlineLimit** в 0, чтобы отключить эту функцию.

- src/components/RawStylesUrl.astro

```
---  
// Расширенный пример! Не  
рекомендуется для  
большинства пользователей!  
import stylesUrl from  
'../styles/main.css?url';  
---  
<link rel="preload" href=  
{stylesUrl} as="style">  
<link rel="stylesheet"  
href={stylesUrl}>
```

Скрипты с : async, defer

В современных сайтах скрипты обычно «тяжелее», чем **HTML**: они весят больше, дольше обрабатываются.

Когда браузер загружает **HTML** и доходит до тега `<script>...</script>`, он не может продолжать строить **DOM**. Он должен сначала выполнить скрипт. То же самое происходит и с внешними скриптами `<script src="..."></script>`: браузер должен подождать, пока загрузится скрипт, выполнить его, и только затем обработать остальную страницу.

Это ведёт к двум важным проблемам:

- Скрипты не видят **DOM**-элементы ниже себя, поэтому к ним нельзя добавить обработчики и т.д.
- Если вверху страницы объёмный скрипт, он «блокирует» страницу.

Пользователи не видят содержимое страницы, пока он не загрузится и не запустится:

```
<p>...содержимое перед  
скриптом...</p>
```

```
<script  
src="https://javascript.info/  
article/script-async-  
defer/long.js?speed=1">  
</script>
```

```
<!-- Это не отобразится,  
пока скрипт не загрузится -  
>
```

```
<p>...содержимое после  
скрипта...</p>
```

Конечно, есть пути, как это обойти.
Например, мы можем поместить скрипт

внизу страницы. Тогда он сможет видеть элементы над ним и не будет препятствовать отображению содержимого страницы:

```
<body>
  ... всё содержимое над
  скриптом...

  <script
src="https://javascript.info
/article/script-async-
defer/long.js?speed=1">
</script>
</body>
```

Но это решение далеко от идеального. Например, браузер замечает скрипт (и может начать загружать его) только после того, как он полностью загрузил **HTML**-документ. В случае с длинными **HTML**-

страницами это может создать заметную задержку.

Такие вещи незаметны людям, у кого очень быстрое соединение, но много кто в мире имеет медленное подключение к интернету или использует не такой хороший мобильный интернет.

К счастью, есть два атрибута тега `<script>`, которые решают нашу проблему: **defer** и **async**.

defer

Атрибут **defer** сообщает браузеру, что он должен продолжать обрабатывать страницу и загружать скрипт в фоновом режиме, а затем запустить этот скрипт, когда **DOM** дерево будет полностью построено.

Вот тот же пример, что и выше, но с **defer**:

```
<p>...содержимое перед  
скриптом...</p>  
  
<script defer  
src="https://javascript.info  
o/article/script-async-  
defer/long.js?speed=1">  
</script>  
  
<!-- отображается сразу же  
-->  
<p>...содержимое после  
скрипта...</p>
```

- Скрипты с **defer** никогда не блокируют страницу.
- Скрипты с **defer** всегда выполняются, когда дерево **DOM**

готово, но до события

DOMContentLoaded.

Следующий пример это показывает:

```
<p>...содержимое до  
скрипта...</p>
```

```
<script>
```

```
document.addEventListener('  
DOMContentLoaded', () =>  
alert("Дерево DOM готово  
после скрипта с  
'defer'!"));  
</script>
```

```
<script defer  
src="https://javascript.info/  
o/article/script-async-  
defer/long.js?speed=1">  
</script> // (2)
```


<p>...содержимое после скрипта...</p>

- Содержимое страницы отобразится мгновенно.
- Событие **DOMContentLoaded** подождёт отложенный скрипт. Оно будет сгенерировано, только когда скрипт (2) будет загружен и выполнен.

Отложенные с помощью **defer** скрипты сохраняют порядок относительно друг друга, как и обычные скрипты.

Допустим, у нас есть два скрипта с **defer**: **small.js** и **long.js**:

```
<script defer  
src="https://javascript.info/article/script-async-
```

```
defer/long.js"></script>  
<script defer  
src="https://javascript.info  
o/article/script-async-  
defer/small.js"></script>
```

Браузеры сканируют страницу на предмет скриптов и загружают их параллельно в целях увеличения производительности. Поэтому и в примере выше оба скрипта скачиваются параллельно. **small.js** скорее всего загрузится первым.

...Но **defer** не только говорит браузеру «не блокировать рендеринг», он также обеспечивает правильную последовательность выполнения скриптов. Даже если **small.js** загрузится первым, он будет ждать выполнения **long.js**.

Это важно в тех случаях, когда нам сначала нужно загрузить **JavaScript**-библиотеку, а затем скрипт, который от неё зависит.

- Атрибут **defer** предназначен только для внешних скриптов. Атрибут **defer** будет проигнорирован, если в теге `<script>` нет **src**.

async

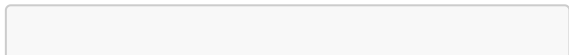
Атрибут **async** означает, что скрипт абсолютно независим:

- Страница не ждёт асинхронных скриптов, содержимое обрабатывается и отображается.
- Событие **DOMContentLoaded** и асинхронные скрипты не ждут друг друга:

- **DOMContentLoaded** может произойти как до асинхронного скрипта (если асинхронный скрипт завершит загрузку после того, как страница будет готова),
- ...так и после асинхронного скрипта (если он короткий или уже содержится в **HTTP**-кеше)
- Остальные скрипты не ждут **async**, и скрипты с **async** не ждут другие скрипты.

Так что если у нас есть несколько скриптов с **async**, они могут выполняться в любом порядке.

То, что первое загрузится – запустится в первую очередь:



<p>...содержимое перед
скриптами...</p>

<script>

```
document.addEventListener('
DOMContentLoaded', () =>
alert("DOM готов!"));
</script>
```

<script async
src="https://javascript.info/article/script-async-defer/long.js"></script>

<script async
src="https://javascript.info/article/script-async-defer/small.js"></script>

<p>...содержимое после
скриптов...</p>

Содержимое страницы отображается сразу же : **async** его не блокирует.

DOMContentLoaded может произойти как до, так и после **async**, никаких гарантий нет. Асинхронные скрипты не ждут друг друга. Меньший скрипт **small.js** идёт вторым, но скорее всего загрузится раньше **long.js**, поэтому и запустится первым. То есть, скрипты выполняются в порядке загрузки. Асинхронные скрипты очень полезны для добавления на страницу сторонних скриптов: счётчиков, рекламы и т.д. Они не зависят от наших скриптов, и мы тоже не должны ждать их:

```
<!-- Типичное подключение  
скрипта Google Analytics --  
>  
<script async  
src="https://google-
```

```
analytics.com/analytics.js"
></script>
```

- Атрибут **async** предназначен только для внешних скриптов. Как и в случае с **defer**, атрибут **async** будет проигнорирован, если в теге `<script>` нет **src**.

Динамически загружаемые скрипты

Мы можем также добавить скрипт и динамически, с помощью **JavaScript**:

```
let script =
document.createElement('scr
ipt');
script.src =
"/article/script-async-
defer/long.js";
```

```
document.body.append(script  
); // (*)
```

Скрипт начнёт загружаться, как только он будет добавлен в документ (*).

Динамически загружаемые скрипты по умолчанию ведут себя как **«async»**.

То есть:

Они никого не ждут, и их никто не ждёт. Скрипт, который загружается первым – запускается первым (в порядке загрузки). Мы можем изменить относительный порядок скриптов с «первый загрузился – первый выполнен» на порядок, в котором они идут в документе (как в обычных скриптах) с помощью явной установки свойства **async** в **false**:


```
let script =  
document.createElement('script');  
script.src =  
"/article/script-async-  
defer/long.js";  
  
script.async = false;  
  
document.body.append(script  
);
```

Например, здесь мы добавляем два скрипта. Без **script.async=false** они запускались бы в порядке загрузки (**small.js** скорее всего запустился бы раньше). Но с этим флагом порядок будет как в документе:

```
function loadScript(src) {  
    let script =  
    document.createElement('scr  
ipt');  
    script.src = src;  
    script.async = false;  
  
    document.body.append(script  
);  
}  
  
// long.js запускается  
первым, так как async=false  
loadScript("/article/script  
-async-defer/long.js");  
loadScript("/article/script  
-async-defer/small.js");
```

Итого у **async** и **defer** есть кое-что общее:
они не блокируют отрисовку страницы.
Так что пользователь может просмотреть

содержимое страницы и ознакомиться с ней сразу же.

Но есть и значимые различия:

Порядок **DOMContentLoaded async**

Порядок загрузки (кто загрузится первым, тот и сработает). Не имеет значения.

Может загрузиться и выполниться до того, как страница полностью загрузится. Такое случается, если скрипты маленькие или хранятся в кеше, а документ достаточно большой. **defer** Порядок документа (как расположены в документе). Выполняется после того, как документ загружен и обработан (ждёт), непосредственно перед **DOMContentLoaded**.

Страница без скриптов должна быть рабочей Пожалуйста, помните, что когда вы используете **defer**, страница видна до того, как скрипт загрузится.

Пользователь может знакомиться с содержимым страницы, читать её, но графические компоненты пока отключены.

Поэтому обязательно должна быть индикация загрузки, нерабочие кнопки – отключены с помощью **CSS** или другим образом. Чтобы пользователь явно видел, что уже готово, а что пока нет.

На практике **defer** используется для скриптов, которым требуется доступ ко всему **DOM** и/или важен их относительный порядок выполнения.

А **async** хорош для независимых скриптов, например счётчиков и рекламы, относительный порядок выполнения которых не играет роли.

Скрипты и обработка событий

Вы можете отправлять **JavaScript** в браузер и добавлять функциональность в компоненты **Astro**, используя `<script>` теги в шаблоне компонента.

Скрипты добавляют интерактивность вашему сайту, например, обрабатывают события или динамически обновляют контент, без необходимости использования фреймворков пользовательского интерфейса, таких как **React, Svelte или Vue**. Это позволяет избежать накладных расходов, связанных с использованием **JavaScript** в фреймворках, и не требует знания каких-либо дополнительных фреймворков для создания полнофункционального веб-сайта или приложения.

Скрипты на стороне клиента

client-side-scripts

Скрипты можно использовать для добавления обработчиков событий, отправки аналитических данных, воспроизведения анимации и всего остального, что **JavaScript** может делать в веб-среде.

Astro автоматически расширяет возможности стандартного **HTML**-`<script>` тега, добавляя поддержку сборки, **TypeScript** и многое другое. Подробнее о том, как **Astro** обрабатывает скрипты, можно узнать [здесь](#).

- `src/components/ConfettiButton.astro`

```
<button data-confetti-  
button>Celebrate!</button>
```

```
<script>
  // Import from npm
  package.

  import confetti from
  'canvas-confetti';

  // Find our component DOM
  on the page.
  const buttons =
  document.querySelectorAll('
  [data-confetti-button]');

  // Add event listeners to
  fire confetti when a button
  is clicked.
  buttons.forEach((button)
  => {

  button.addEventListener('click', () => confetti());
  });
</script>
```

Узнайте, **когда ваши скрипты не будут обрабатываться**, чтобы устранить неполадки в их работе или чтобы узнать, как намеренно отказаться от этой обработки.

Обработка скриптов

По умолчанию **Astro** обрабатывает `<script>` теги, не содержащие атрибутов (кроме **src**), следующим образом:

- Поддержка **TypeScript**: Все скрипты по умолчанию написаны на **TypeScript**.
- Импорт и объединение файлов: импорт локальных файлов или модулей **npm**, которые будут объединены в один пакет.

- Тип модуля: Обработанные скрипты обрабатываются **type="module"** автоматически.
- Дедупликация: Если компонент, содержащий скрипт, **<script>** используется на странице несколько раз, скрипт будет включен только один раз.
- Автоматическое встраивание: если скрипт достаточно мал, **Astro** встроит его непосредственно в **HTML**-код, чтобы уменьшить количество запросов.
- `src/components/Example.astro`

```
<script>  
  // Processed! Bundled!  
  TypeScript!  
  // Importing local
```

```
scripts and from npm  
packages works.  
</script>
```

Необработанные скрипты

Astro не будет обрабатывать `<script>` тег, если у него есть какой-либо атрибут, кроме **src**.

Вы можете добавить **is:inline** директиву, чтобы намеренно отказаться от обработки скрипта.

- `src/components/InlineScript.astro`

```
<script is:inline>  
  // Will be rendered into  
  the HTML exactly as  
  written!  
  // Not transformed: no
```

```
TypeScript and no import  
resolution by Astro.
```

```
// If used inside a  
component, this code is  
duplicated for each  
instance.
```

```
</script>
```

Включите файлы JavaScript на свою страницу.

Возможно, вам потребуется писать скрипты в отдельных **.js** файлах **.ts** или ссылаться на внешний скрипт на другом сервере. Это можно сделать, указав ссылки на них в атрибуте **<script>** тега **src**.

Импорт локальных скриптов

Когда это использовать: когда ваш скрипт находится внутри **src/**.

Astro обработает эти сценарии в соответствии с правилами обработки сценариев.

- `src/components/LocalScripts.astro`

```
<!-- relative path to
script at
`src/scripts/local.js` -->
<script
src="../../../scripts/local.js">
</script>
```

```
<!-- also works for local
TypeScript files -->
<script src="../../../script-with-
types.ts"></script>
```

Загрузка внешних скриптов

Когда это использовать: когда ваш **JavaScript**-файл находится внутри **public/** или на **CDN**.

Чтобы загружать скрипты вне папки вашего проекта **src/**, добавьте **is:inline** директиву. Такой подход позволяет избежать обработки, объединения и оптимизации **JavaScript**, которые **Astro** предоставляет при импорте скриптов, как описано выше.

- `src/components/ExternalScripts.astro`

```
<!-- absolute path to a
script at `public/my-
script.js` -->
<script is:inline src="/my-
script.js"></script>
```

```
<!-- full URL to a script
on a remote server -->
<script is:inline
src="https://my-
analytics.com/script.js">
</script>
```

Общие шаблоны скриптов

Обработка onclick и другие события

Некоторые **UI**-фреймворки используют собственный синтаксис для обработки событий, например **onClick={...}** (**React/Preact**) или **@click="..."** (**Vue**). **Astro** же более точно следует стандарту **HTML** и не использует собственный синтаксис для событий.

Вместо этого вы можете использовать **addEventListener** тег **<script>** для

обработки взаимодействия с пользователем.

- src/components/AlertButton.astro

```
<button class="alert">Click me!</button>
```

```
<script>
  // Find all buttons with
  the `alert` class on the
  page.
  const buttons =
    document.querySelectorAll('
    button.alert');

  // Handle clicks on each
  button.
  buttons.forEach((button)
    => {

    button.addEventListener('cl
    ick', () => {
```

```
        alert('Button was  
clicked!');  
    });  
});  
</script>
```

Если на странице несколько `<AlertButton />` компонентов, **Astro** не будет запускать скрипт несколько раз. Скрипты объединены в пакет и включаются только один раз на страницу. Использование **querySelectorAll** гарантирует, что этот скрипт прикрепит обработчик событий к каждой кнопке с **alert** классом, найденным на странице.

Веб-компоненты с пользовательскими элементами

С помощью стандарта **Web Components** вы можете создавать собственные **HTML-**

элементы с настраиваемым поведением. Определение пользовательского элемента в **.astro** компоненте позволяет создавать интерактивные компоненты без необходимости использования библиотеки **UI**-фреймворка.

В этом примере мы определяем новый `<astro-heart>` **HTML**-элемент, который отслеживает, сколько раз вы нажимаете на кнопку с сердечком, и обновляет его `<span>` последним значением.

- `src/components/AstroHeart.astro`

```
<!-- Wrap the component
elements in our custom
element "astro-heart". -->
<astro-heart>
  <button aria-
label="Heart">❤️</button> ×
  <span>0</span>
```

```
</astro-heart>
```

```
<script>
```

```
  // Define the behaviour  
  for our new type of HTML  
  element.
```

```
  class AstroHeart extends  
  HTMLElement {  
    connectedCallback() {  
      let count = 0;  
  
      const heartButton =  
      this.querySelector('button')  
    );  
      const countSpan =  
      this.querySelector('span');  
  
      // Each time the  
      button is clicked, update  
      the count.  
  
      heartButton.addEventListener('click', () => {  
        count++;
```

```
countSpan.textContent =  
count.toString();  
});  
}  
}  
  
// Tell the browser to  
use our AstroHeart class  
for <astro-heart> elements.  
  
customElements.define('astro-heart', AstroHeart);  
</script>
```

Использование пользовательского элемента здесь имеет два преимущества:

- Вместо поиска по всей странице с помощью `<p>` **`document.querySelector()`**, можно использовать `<p>`

this.querySelector(), который ищет только внутри текущего экземпляра пользовательского элемента. Это упрощает работу только с дочерними элементами одного экземпляра компонента за раз.

- Хотя метод `<script> await` выполняется только один раз, браузер будет запускать **connectedCallback()** метод нашего пользовательского элемента каждый раз, когда обнаружит его `<astro-heart>` на странице. Это означает, что вы можете безопасно писать код для одного компонента за раз, даже если вы планируете использовать этот компонент несколько раз на странице.

Более подробную информацию о пользовательских элементах можно найти [в руководстве по многократно используемым веб-компонентам на web.dev](#) и [во вводной статье о пользовательских элементах на MDN](#).

Передайте переменные метаданных скриптам.

В компонентах **Astro** код, находящийся в метаданных (между `---` блоками), выполняется на сервере и недоступен в браузере.

Для передачи переменных на стороне сервера в клиентские скрипты, сохраняйте их в ***data-**** атрибутах **HTML**-элементов. Затем скрипты смогут получить доступ к этим значениям,

используя соответствующее **dataset** свойство.

В этом примере компонента **message** свойство хранится в **data-message** атрибуте, чтобы пользовательский элемент мог прочитать **this.dataset.message** и получить значение этого свойства в браузере.

- `src/components/AstroGreet.astro`

```
---  
const { message = 'Welcome,  
world!' } = Astro.props;  
---  
  

```

```
</astro-greet>
```

```
<script>
```

```
  class AstroGreet extends  
    HTMLElement {  
    connectedCallback() {  
      // Read the message  
      from the data attribute.  
      const message =  
this.dataset.message;  
      const button =  
this.querySelector('button'  
);  
  
      button.addEventListener('cl  
ick', () => {  
        alert(message);  
      });  
    }  
  }  
  
  customElements.define('astr
```

```
o-greet', AstroGreet);  
</script>
```

Теперь мы можем использовать наш компонент несколько раз, и каждый раз будем получать разное сообщение.

- src/pages/example.astro

```
---  
import AstroGreet from  
'../components/AstroGreet.astro';  
---  
  
<!-- Use the default  
message: "Welcome, world!"  
-->  
<AstroGreet />  
  
<!-- Use custom messages  
passed as a props. -->
```



```
<AstroGreet message="Lovely  
day to build components!"  
/>  
<AstroGreet message="Glad  
you made it! 🙌" />
```

- Вы знали?

На самом деле, именно это **Astro** делает за кулисами, когда вы передаете свойства компоненту, написанному с использованием UI-фреймворка, такого как **React!** Для компонентов с директивой **client:*** **Astro** создает `<astro-island>` пользовательский элемент с **props** атрибутом, который хранит ваши серверные свойства в **HTML**-выводе.

Сочетание скриптов и **UI**-фреймворков
Элементы, отображаемые **UI**-
фреймворком, могут быть недоступны на
момент `<script>` выполнения тега. Если
вашему скрипту также необходимо
обрабатывать компоненты **UI**-
фреймворка , рекомендуется
использовать пользовательский элемент.

markdown-content

hydration

Термин "гидратироваться" (hydrate /
hydration) в контексте Astro и
современных фреймворков (таких как
React, Vue, Svelte) означает процесс
превращения статического,
неинтерактивного HTML-кода, который
был сгенерирован на сервере, в
полностью интерактивный, работающий

клиентский компонент с помощью JavaScript.

Вот подробное объяснение того, что подразумевается под этим словом в приведенных вами примерах:

Что такое "гидратация" (Hydration)? В мире разработки веб-приложений гидратация — это ключевой этап, следующий за серверным рендерингом (SSR) или генерацией статических сайтов (SSG).

Представьте, что вы строите дом. Сначала вы возводите каркас и стены (это статический HTML, который виден сразу). Затем вы подключаете электричество, водопровод и устанавливаете розетки и выключатели (это JavaScript, который делает дом функциональным).

Гидратация — это процесс подключения "электричества" к вашему HTML.

Как это работает в примере с Astro

В вашем примере: `<HeavyImageCarousel
client:visible={{rootMargin: "200px"}} />`

На сервере: Когда пользователь запрашивает страницу, Astro отправляет готовый, но неактивный HTML-код для HeavyImageCarousel. Пользователь видит изображение-заглушку или статический макет карусели сразу, но кнопки "вперед/назад" пока не работают. JavaScript, необходимый для их работы, еще не загружен или не запущен.

На клиенте (в браузере): Когда браузер пользователя загружает JavaScript-файл для этого компонента и видит директиву `client:visible`, он начинает процесс гидратации.

Гидратация: JavaScript берет существующий HTML, "прикрепляет" к нему все обработчики событий (слушатели кликов, свайпов) и инициализирует внутреннее состояние компонента.

Теперь карусель становится полностью функциональной и интерактивной.

Что означает `client:visible` и `rootMargin` в этом контексте

Директива `client:visible` указывает Astro отложить процесс гидратации до тех пор, пока компонент не попадет в поле зрения пользователя (вьюпорт). Это техника называется "ленивой гидратацией" (lazy hydration) или "частичной гидратацией" (partial hydration).

Слово "гидратироваться" здесь означает:

"Запустить и применить клиентский JavaScript к этому компоненту, чтобы сделать его интерактивным".

Пример с `rootMargin`:

`<HeavyImageCarousel client:visible />`:

Компонент гидратируется (становится активным), когда его верхний край достигает видимой части экрана.

`<HeavyImageCarousel client:visible={{rootMargin: "200px"}} />`: Компонент гидратируется (становится активным), когда до его появления на экране остается еще 200 пикселей прокрутки.

Зачем это нужно?

Как указано в вашем тексте, это делается для оптимизации:

Уменьшение CLS (Cumulative Layout Shift):
Компонент загружается заранее и не вызывает резких "прыжков" страницы при появлении.

Ускорение загрузки страницы: Браузеру не нужно загружать и запускать тяжелый JavaScript сразу для всех компонентов страницы. Он гидратирует только те, которые скоро понадобятся пользователю.

По сути, "гидратироваться" — это технический термин, описывающий оживление статической веб-страницы с помощью JavaScript.

XSS

межсайтовый скриптинг

Материалы для статьи

[Астро документация](#)

[set:directives](#)

[Astro-syntax](#)

[dynamic-tags](#)

[XSS](#)

[Framework-components](#)

[passing-components-to-mdx-content](#)

[requestIdleCallback](#)

[requestidlecallback-method](#)

[addclientdirective-option](#)

server-islands

global-styles

defer

client-side-scripts

markdown-content